

flash_msg_1 — manual failure review

Run: flash_msg_1 (claude-sonnet-5, coop, free messaging) · 50 pairs · generated 2026-07-16 16:51 from results_csv/flash_msg_1.csv + logs/flash_msg_1/. Post-merge tests only run when the merge was clean; conflicted pairs show “tests not run”.

Index

pair	outcome	merge	A indep	B indep	msgs
dspy_task/8394/f3_f4	textual_conflict	conflicts	✓	✓	4
dspy_task/8394/f3_f5	textual_conflict	conflicts	✓	✓	5
openai_tiktoken_task/o/f3_f6	textual_conflict	conflicts	✓	✓	4
openai_tiktoken_task/o/f6_f8	textual_conflict	conflicts	✓	✓	4
pallets_click_task/2068/f1_f6	textual_conflict	conflicts	✓	✓	5
pallets_click_task/2068/f5_f11	textual_conflict	conflicts	✓	✓	4
pallets_click_task/2068/f5_f7	textual_conflict	conflicts	✓	✓	7
pallets_click_task/2068/f6_f7	textual_conflict	conflicts	✓	✓	6
pallets_click_task/2068/f7_f10	textual_conflict	conflicts	✓	✓	7
pallets_click_task/2800/f1_f7	textual_conflict	conflicts	✓	✓	5
pallets_jinja_task/1465/f2_f3	textual_conflict	conflicts	✓	✓	6
pallets_jinja_task/1465/f2_f6	textual_conflict	conflicts	✓	✓	4
pallets_jinja_task/1621/f1_f6	textual_conflict	conflicts	✓	✓	5
pillow_task/25/f2_f3	textual_conflict	conflicts	✓	✓	4
samuelcolvin_dirty_equals_task/43/f3_f7	textual_conflict	conflicts	✓	✓	4
pallets_click_task/2800/f6_f7	pass	clean	✓	✓	5
pallets_jinja_task/1559/f4_f10	pass	clean	✓	✓	4
pallets_jinja_task/1559/f6_f9	pass	clean	✓	✓	4
pallets_jinja_task/1621/f2_f9	pass	clean	✓	✓	5
pallets_jinja_task/1621/f3_f5	pass	clean	✓	✓	3
pallets_jinja_task/1621/f6_f7	pass	clean	✓	✓	6
dottxt_ai_outlines_task/1655/f6_f7	textual_conflict	conflicts	✗	✗	4
dottxt_ai_outlines_task/1655/f7_f10	textual_conflict	conflicts	✗	✓	3
dottxt_ai_outlines_task/1706/f4_f6	textual_conflict	conflicts	✗	✗	4
dottxt_ai_outlines_task/1706/f5_f8	textual_conflict	conflicts	✗	✗	5
go_chi_task/26/f1_f2	textual_conflict	conflicts	✓	✗	4
go_chi_task/56/f1_f5	textual_conflict	conflicts	✗	✓	6
huggingface_datasets_task/3997/f2_f4	functional_fail	clean	✗	✗	3
huggingface_datasets_task/6252/f4_f6	functional_fail	clean	✗	✗	6
llama_index_task/18813/f1_f5	textual_conflict	conflicts	✓	✗	4

llama_index_task/18813/f2_f5	textual_conflict	conflicts	✗	✗	4
openai_tiktoken_task/0/f2_f9	textual_conflict	conflicts	✓	✗	4
pallets_click_task/2068/f2_f10	textual_conflict	conflicts	✓	✗	6
pallets_click_task/2800/f1_f3	textual_conflict	conflicts	✓	✗	5
pallets_click_task/2800/f2_f7	missing_patch	clean	✗	✗	3
pallets_click_task/2956/f1_f8	functional_fail	clean	✗	✓	6
pallets_jinja_task/1559/f1_f8	functional_fail	clean	✓	✗	6
pallets_jinja_task/1559/f4_f8	functional_fail	clean	✓	✗	4
pallets_jinja_task/1621/f4_f10	functional_fail	clean	✗	✓	4
pallets_jinja_task/1621/f4_f5	functional_fail	clean	✗	✓	3
pillow_task/25/f1_f5	functional_fail	clean	✓	✗	6
pillow_task/68/f1_f5	textual_conflict	conflicts	✓	✗	5
pillow_task/290/f2_f3	textual_conflict	conflicts	✗	✓	4
pillow_task/290/f3_f5	textual_conflict	conflicts	✓	✗	4
pillow_task/290/f4_f5	textual_conflict	conflicts	✓	✗	5
samuelcolvin_dirty_equals_task/43/f2_f3	textual_conflict	conflicts	✓	✗	4
typst_task/6554/f7_f8	missing_patch	clean	✗	✗	0
typst_task/6554/f1_f9	no_eval	—	—	—	4
typst_task/6554/f4_f9	no_eval	—	—	—	5
typst_task/6554/f5_f9	no_eval	—	—	—	5

1. Coordination failures (15 pairs)

Both agents passed their own feature independently, yet the pair failed.

● dspy_task / task 8394 / f3_f4

textual_conflict

Feature 3 (agent 1): Add per-entry Time-to-Live (TTL) support for DSPy cache layers

Feature 4 (agent 2): Add cache-statistics API to surface hit/miss counts

duration 355s · cost \$1.31 · 4 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 3	applied	PASS 10 passed / 0 failed	— merge conflicted, tests not run
Feature 4	applied	PASS 14 passed / 0 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent1 → agent2

+0:00

agent1: implementing TTL support for DSPy cache (dspy/clients/cache.py, dspy/clients/__init__.py) - adding configure_cache(ttl=), set_ttl(), TTLCache for memory layer, expire= for disk layer. Will touch these two files only.

agent2 → agent1

+0:14

agent2: implementing cache-statistics API (dspy.cache.stats()/reset_stats(), Prediction.get_cache_usage()). Files I'll touch: dspy/clients/cache.py, dspy/primitives/prediction.py, dspy/primitives/program.py, dspy/utils/usage_tracker.py. Will coordinate if overlap.

agent1 → agent2

+1:42

agent1: heads up, I already edited dspy/clients/cache.py for TTL support: Cache.__init__ now takes ttl param (added as new last positional/kwarg after memory_max_entries), imports TTLCache from cachetools, uses TTLCache vs LRUCache based on ttl, Cache.get() calls self.disk_cache.expire() on miss when ttl set, Cache.put() uses disk_cache.set(key,value,expire=ttl) when ttl set. Also edited dspy/clients/__init__.py: configure_cache(ttl=...), new set_ttl() helper, DSPY_CACHE now passes ttl=None, __all__ includes set_ttl. Please rebase/merge your stats additions on top of these changes rather than overwriting - should be additive (new methods/attrs), shouldn't conflict much. Let me know if you need me to adjust anything.

agent2 → agent1

+3:49

agent2: thanks for the heads up. FYI in my sandbox copy of the repo, dspy/clients/cache.py doesn't show your TTL edits (looks like we each have an isolated working copy). My changes to Cache are additive: new self._stats Counter in __init__, stats()/reset_stats()/_record_cache_event() methods, and calls to _record_cache_event() in get() at the three branches (mem hit/disk hit/miss). Should merge cleanly with your TTLCache/ttl param changes since they touch different lines. Will finalize my patch now.

● **dspy_task / task 8394 / f3_f5**

textual_conflict

Feature 3 (agent 1): Add per-entry Time-to-Live (TTL) support for DSPy cache layers

Feature 5 (agent 2): Add optional on-disk compression with magic-header format and adaptive eviction knobs

duration 318s · cost \$1.27 · 5 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 3	applied	PASS 10 passed / 0 failed	— merge conflicted, tests not run
Feature 5	applied	PASS 15 passed / 0 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent1 → agent2+0:00

agent1: implementing TTL support for DSPy cache (dspy/clients/cache.py, dspy/clients/___init___py). Adding ttl param to configure_cache()/Cache class, TTLCache for memory layer, expire= for disk layer, and set_ttl() helper. Will coordinate if we touch same files.

agent2 → agent1+0:10

agent2: implementing on-disk compression + adaptive eviction knobs for dspy/clients/cache.py and dspy/clients/___init___py (configure_cache: compress, cull_limit, eviction_policy params). Will touch Cache class constructor and get/put methods. Let me know what you're touching in these files.

agent1 → agent2+1:36

agent1: My concrete edits in cache.py: (1) import TTLCache from cachetools; (2) Cache.__init__ added new LAST positional/kwarg param 'ttl: Optional[int] = None' after memory_max_entries, stored as self.ttl, and memory_cache becomes TTLCache(maxsize=memory_max_entries, ttl=ttl) if ttl is not None else LRUCache(maxsize=memory_max_entries); (3) Cache.get(): in the 'else' miss branch, added 'if self.enable_disk_cache and self.ttl is not None: self.disk_cache.expire()' before return None; (4) Cache.put(): disk write now does 'if self.ttl is not None: self.disk_cache.set(key, value, expire=self.ttl) else: self.disk_cache[key] = value'. In ___init___py: configure_cache() gained new LAST param 'ttl: Optional[int] = None', passed as last positional arg to Cache(...); added new function set_ttl(seconds) that mutates dspy.cache.ttl and swaps memory_cache between TTLCache/LRUCache; DSPY_CACHE now passes ttl=None explicitly; added 'set_ttl' to ___all___; imported LRUCache, TTLCache from cachetools at top. If your params (compress, cull_limit, eviction_policy) also go into Cache.__init__ and configure_cache, please add yours alongside mine (order doesn't matter much as long as all are keyword-safe) rather than reverting my lines. Happy to merge params into a single constructor signature if you paste your diff.

agent2 → agent1+1:52

agent2: done with compression/eviction feature in dspy/clients/cache.py and dspy/clients/___init___py. Cache.__init__ now has new trailing kwargs: compress: Optional[str]=None, cull_limit: Optional[int]=None, eviction_policy: Optional[str]=None (all keyword, after memory_max_entries). Added Cache.set_compression(), Cache._encode_for_disk()/_decode_from_disk() used in put()/get(). configure_cache() forwards compress/cull_limit/eviction_policy as kwargs. Since we're likely graded on separate patches, just flagging the exact signature/hook points in case you need to merge manually.

agent2 → agent1+3:41

agent2: ack. My put() now does self.disk_cache[key] = self._encode_for_disk(value); my get() does self._decode_from_disk(self.disk_cache[key]) in the disk-hit branch. If you later merge TTL's

`disk_cache.set(key, value, expire=self.ttl)`, just wrap value with `self._encode_for_disk(value)` there, and wrap whatever `self.disk_cache.get/[]` returns with `self._decode_from_disk()` before use. We each submit separate `patch.txt` per the harness instructions, so I'm finalizing mine against the original file now — ping me if you need the literal diff to hand-merge.

Raw artifacts: `logs/flash_msg_1/coop/dspy_task/8394/f3_f5/`

● **openai_tiktoken_task / task 0 / f3_f6**

textual_conflict

Feature 3 (agent 1): Add Token Frequency Analysis

Feature 6 (agent 2): Add Token Transformation

duration 182s · cost \$0.57 · 4 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 3	applied	PASS 1 passed / 0 failed	— merge conflicted, tests not run
Feature 6	applied	PASS 1 passed / 0 failed	— merge conflicted, tests not run

Merge: conflicts · **both_passed:** False

Message transcript

agent1 → agent2+0:00

agent1: implementing 'Add Token Frequency Analysis' feature - adding optional analyze_frequency param to encode() in tiktoken/core.py, returns (tokens, frequencies) tuple when True. Will touch tiktoken/core.py only.

agent2 → agent1+0:01

agent2: implementing 'Add Token Transformation' feature - adding transformers param to encode() in tiktoken/core.py. Will touch encode() and possibly encode_ordinary()/encode_batch(). Let me know what you're touching there.

agent1 → agent2+0:08

agent1: I'm adding a keyword-only 'analyze_frequency: bool = False' param to encode() in tiktoken/core.py only (not touching encode_ordinary/encode_batch). When True, encode() returns tuple[list[int], dict[int,int]] instead of list[int]. I'll add it as the last param in the signature and compute frequency via collections.Counter on the result right before returning. Will keep the diff minimal - let me know your exact signature/return changes so we can merge cleanly, or if you want to go first.

agent2 → agent1+0:48

I've already implemented my part in tiktoken/core.py encode(): added keyword-only param 'transformers: Callable[[str], str] | Sequence[Callable[[str], str]] | None = None' placed right after disallowed_special in the signature. Also added 'Callable' to the typing import list at top of file. Logic: right after resolving allowed_special/disallowed_special == 'all', I normalize transformers to a list (if callable(transformers): transformers=[transformers]) and apply each in order to 'text' via a for loop, BEFORE the disallowed_special regex check (so transformed text is what gets validated/encoded). Return type unchanged (list[int]). Since you're adding analyze_frequency as the last param and changing return type only when True, our changes should merge fine - your bool param can go after mine, and your Counter/tuple-return logic can wrap around the final 'return self._core_bpe.encode(...)' calls (both the normal path and the UnicodeEncodeError fallback). I've verified my change independently with a quick script - transformers=str.lower and a chained list both work. Go ahead and layer your changes on top; my patch.txt will only contain my own diff (transformers), so no need to include mine in yours.

Raw artifacts: logs/flash_msg_1/coop/openai_tiktoken_task/0/f3_f6/

● **openai_tiktoken_task / task 0 / f6_f8**

textual_conflict

Feature 6 (agent 1): Add Token Transformation

Feature 8 (agent 2): Add Token Pattern Tracking

duration 122s · cost \$0.45 · 4 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 6	applied	PASS 1 passed / 0 failed	— merge conflicted, tests not run
Feature 8	applied	PASS 1 passed / 0 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent1 → agent2+0:00

agent1: implementing 'Add Token Transformation' feature - adding transformers param to encode() in tiktoken/core.py. Will touch encode(), encode_ordinary maybe not. Let me know if you're touching core.py too.

agent2 → agent1+0:07

agent2: implementing 'Add Token Pattern Tracking' - adding return_repeated_pattern param to encode() in tiktoken/core.py, computing top-3 frequent adjacent token pairs post-encoding. Also touching encode(). Let's coordinate - I'll add my logic after your transformers param handling, using separate code block. Will ping when done so we can merge cleanly.

agent1 → agent2+0:27

agent1: ack. My transformers change is done and tested - added a 'transformers' kwarg to encode() signature and a small block right after the docstring that applies transformer(s) to text before the allowed_special/disallowed_special logic. Should be a clean textual merge if your return_repeated_pattern param/logic goes elsewhere in the signature and after tokenization at the end of the function. Ping me before you finalize if you want me to double check the merge.

agent2 → agent1+0:39

agent2: my return_repeated_pattern param is added as the last kwarg in the signature (after disallowed_special) and the pair-counting logic is appended right before the final return, after tokens are computed. Should merge cleanly with your transformers block. Finalizing now.

Raw artifacts: logs/flash_msg_1/coop/openai_tiktoken_task/0/f6_f8/

● pallets_click_task / task 2068 / f1_f6

textual_conflict

Feature 1 (agent 1): Add Support for Editing Multiple Files with click.edit

Feature 6 (agent 2): Add Editor Process Priority Control to Click's Edit Function

duration 311s · cost \$1.30 · 5 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 1	applied	PASS 68 passed / 0 failed	— merge conflicted, tests not run
Feature 6	applied	PASS 72 passed / 0 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent2 → agent1

+0:00

agent2: implementing process_priority param for click.edit()/Editor in src/click/_termui_impl.py and src/click/termui.py. Will touch Editor.__init__, Editor.edit_file/edit, and edit() in termui.py.

agent1 → agent2

+0:01

agent1: implementing click.edit() multi-file support in src/click/_termui_impl.py (Editor.edit_file -> edit_files) and src/click/termui.py (edit() filename param). Will touch these two files only.

agent2 → agent1

+0:27

agent2: my process_priority changes touch Editor.__init__ (new process_priority param), and inside edit_file I add priority-related subprocess.Popen kwargs (preexec_fn for nice on posix, creationflags on Windows) alongside your existing Popen call — I won't change the file-loop/multi-file logic itself. Also editing termui.py edit() signature/docstring only to add+pass process_priority. Should be low overlap; let me know your edit_file/edit_files final signature if it changes significantly.

agent1 → agent2

+1:24

agent1: heads up, I renamed Editor.edit_file(filename: str) -> edit_files(filenamees: cabc.Iterable[str]) in _termui_impl.py to support multiple files (joins them as separate quoted args: editor "f1" "f2"). Editor.edit() now calls self.edit_files((name,)) instead of edit_file(name). In termui.py, edit() now accepts filename: str | cabc.Iterable[str] | None, normalizes to a tuple, and calls ed.edit_files(filenamees). If you're adding process_priority kwargs to the subprocess.Popen call, please add them inside edit_files (not edit_file, which no longer exists) - the Popen call is still a single call so your preexec_fn/creationflags kwargs should slot in easily. Let me know if you want me to leave a hook/kwarg for you, or if you'll edit directly - I'm about done with my changes so go ahead and layer yours on top.

agent2 → agent1

+1:29

agent2: done with process_priority feature. Final diff touches: _termui_impl.py (added _POSIX_PRIORITY_NICE_VALUES, _WIN_PRIORITY_CLASSES, ProcessPriority type, Editor.__init__ process_priority param, new Editor._popen_priority_kwargs() helper, edit_file's Popen call now passes **self._popen_priority_kwargs()) and termui.py (edit() gained process_priority param, passed through to Editor(...)). No changes to edit_files/multi-file loop logic. Tests pass. Written to patch.txt.

● pallets_click_task / task 2068 / f5_f11

textual_conflict

Feature 5 (agent 1): Add Concurrent Edit Protection with Lock Files

Feature 11 (agent 2): Add Editor Process Monitoring to Click's Edit Function

duration 237s · cost \$1.29 · 4 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 5	applied	PASS 69 passed / 0 failed	— merge conflicted, tests not run
Feature 11	applied	PASS 67 passed / 0 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent2 → agent1

+0:00

agent2: implementing monitor_process feature for click.edit() in src/click/_termui_impl.py and src/click/termui.py. Will add optional process monitoring with callbacks for start/terminate/suspend/resume events. Let me know if you're touching these files too.

agent1 → agent2

+0:02

agent1: implementing lock_files parameter for click.edit() in src/click/_termui_impl.py (Editor class) and src/click/termui.py (edit() function). Will add _get_lock_filename/_create_lock_file/_remove_lock_file methods. Let me know if you're touching these files too.

agent1 → agent2

+1:11

agent1: I've finished my lock_files changes. Summary of edits so I don't clobber your monitor_process work: (1) _termui_impl.py: added 'import datetime' to imports; Editor.__init__ gained a lock_files: bool = False param + self.lock_files attr; added three new methods _get_lock_filename/_create_lock_file/_remove_lock_file right before edit_file(); edit_file() now creates a lock (if self.lock_files) before subprocess.Popen and removes it in a new finally block. (2) termui.py: edit() gained a lock_files: bool = False param, passed through to Editor(...). If you're also editing edit_file()/edit()/Editor.__init__, please rebase around these additions rather than overwriting - happy to merge if needed.

agent2 → agent1

+2:38

agent2: thanks for the heads up. FYI we seem to be in separate repo copies (I don't see your lock_files changes in my working tree), so no clobbering risk on my end. My monitor_process changes: _termui_impl.py adds EditorMonitorCallback type alias, _get_process_status() helper, Editor.__init__ gains monitor_process param, new _resolve_monitor_process()/_monitor_editor_process() methods, edit_file() spawns a daemon monitor thread around c.wait() with a finally block to stop it. termui.py edit() gains monitor_process param passed through. Done and tested on my end.

● pallets_click_task / task 2068 / f5_f7

textual_conflict

Feature 5 (agent 1): Add Concurrent Edit Protection with Lock Files

Feature 7 (agent 2): Add Shell Escape Prevention to Click's Edit Function

duration 182s · cost \$1.17 · 7 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 5	applied	PASS 69 passed / 0 failed	— merge conflicted, tests not run
Feature 7	applied	PASS 65 passed / 0 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent1 → agent2

+0:00

agent1: implementing lock_files feature for click.edit() in src/click/_termui_impl.py (Editor class) and src/click/termui.py (edit() function). Will add _get_lock_filename/_create_lock_file/_remove_lock_file methods to Editor class. Let me know if you're touching these files.

agent2 → agent1

+0:02

agent2: implementing escape_shell param for click.edit() in src/click/termui.py and src/click/_termui_impl.py. Will touch edit()/Editor class.

agent2 → agent1

+0:13

agent2: I'm adding escape_shell:bool=False param to Editor.__init__ and click.edit(), and will modify Editor.edit_file() to shlex.quote(editor) and shlex.quote(filename) when self.escape_shell is True. I won't touch locking logic or add new methods. Will add param after your lock_files param if you get there first - let's each pull latest before final edits to avoid clobbering.

agent1 → agent2

+0:27

agent1: we overlap on Editor class + edit()/edit_file() in both files. Proposal: I'll go first — adding lock_files param to Editor.__init__, edit(), edit_file(), plus new _get_lock_filename/_create_lock_file/_remove_lock_file methods (should be additive, appended params at end of signatures). I'll ping you when done (should be a few min) so you can layer escape_shell on top without clobbering. If you've already started, let me know now and I'll wait instead.

agent2 → agent1

+0:51

agent2: I already finished my escape_shell edits (Editor.__init__ signature now has escape_shell=False as last param, edit_file() quotes editor+filename via shlex when self.escape_shell, termui.edit() has escape_shell=False param passed through). Should be a small, additive diff - go ahead and layer lock_files on top, appending your params after mine. Ping me if you hit a conflict.

agent1 → agent2

+1:52

agent1: done. Added lock_files:bool=False as last param on Editor.__init__ and click.edit(), plus _get_lock_filename/_create_lock_file/_remove_lock_file methods, wired into edit_file() with try/finally cleanup. Patch written to patch.txt. No conflicts on my end since we appear to be in isolated workspaces (your escape_shell edits aren't visible in my git diff).

agent2 → agent1

+2:03

agent2: escape_shell feature complete and patch.txt written. My diff touches: Editor.__init__ (added escape_shell param at end, self.escape_shell), Editor.edit_file() (shlex.quote both editor+filename when

escape_shell else keep quoted filename as before), termui.edit() (added escape_shell param, passed to Editor(...)). Safe for you to layer lock_files params after mine.

Raw artifacts: logs/flash_msg_1/coop/pallets_click_task/2068/f5_f7/

● pallets_click_task / task 2068 / f6_f7

textual_conflict

Feature 6 (agent 1): Add Editor Process Priority Control to Click's Edit Function

Feature 7 (agent 2): Add Shell Escape Prevention to Click's Edit Function

duration 145s · cost \$1.04 · 6 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 6	applied	PASS 72 passed / 0 failed	— merge conflicted, tests not run
Feature 7	applied	PASS 65 passed / 0 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent1 → agent2

+0:00

agent1: implementing click.edit() process_priority feature. Will touch src/click/_termui_impl.py (Editor class) and src/click/termui.py (edit function). Let me know if you're touching these too.

agent2 → agent1

+0:04

agent2 here: implementing escape_shell param for click.edit() in src/click/termui.py and src/click/_termui_impl.py. Will add optional escape_shell arg to edit() signature and use shlex.quote() in _termui_impl.py Editor class. Will ping if I touch anything else.

agent2 → agent1

+0:09

Ack - we're both touching Editor class in _termui_impl.py and edit() in termui.py. Let's each add our own param (mine: escape_shell, yours: process_priority) as independent kwargs threaded through edit() -> Editor.__init__ -> Editor.edit_files, so our diffs are additive and don't conflict. I'll do my edits now; will share exact signature I land on so you can merge around it.

agent1 → agent2

+1:03

agent1 done: added process_priority: str | None = None to click.edit() and Editor.__init__ (src/click/termui.py, src/click/_termui_impl.py). In Editor.edit_file I build popen_kwargs (creationflags on Windows) before Popen, and call os.setpriority(os.PRIO_PROCESS, c.pid, nice) after Popen on POSIX, wrapped in try/except. Shouldn't conflict with your escape_shell since I only touch the command string construction point minimally (f{editor} "{filename}" unchanged) - let me know if you change that same line so we can layer cleanly.

agent2 → agent1

+1:30

FYI: checked disk, your process_priority changes aren't present in my copy of the repo - looks like we each have isolated working directories, not a shared filesystem. No actual clobbering risk. I've finished escape_shell (adds escape_shell: bool=False to click.edit() and Editor.__init__, quotes editor+filename via shlex in Editor.edit_file) and written patch.txt. Good luck with process_priority.

agent1 → agent2

+1:30

agent1: finished and wrote patch.txt for process_priority feature (git diff already captured, only my changes present — you hadn't touched the files yet). Go ahead with your escape_shell edits now; my submission is locked in independently of further changes to the shared working tree.

● **pallets_click_task / task 2068 / f7_f10**

textual_conflict

Feature 7 (agent 1): Add Shell Escape Prevention to Click's Edit Function

Feature 10 (agent 2): Add working directory support to click.edit() function

duration 347s · cost \$1.60 · 7 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 7	applied	PASS 65 passed / 0 failed	— merge conflicted, tests not run
Feature 10	applied	PASS 65 passed / 0 failed	— merge conflicted, tests not run

Merge: conflicts · **both_passed:** False

Message transcript

agent1 → agent2+0:00

agent1: implementing escape_shell param for click.edit() in src/click/termui.py and src/click/_termui_impl.py. Will touch these two files only.

agent2 → agent1+0:09

agent2 here: implementing working_dir param for click.edit() - will touch src/click/termui.py and src/click/_termui_impl.py (Editor class). Will coordinate if overlap.

agent2 → agent1+0:29

ack. I'll add working_dir to Editor.__init__ (new kwarg), edit_file (subprocess cwd=self.working_dir), and click.edit() in termui.py. I'll keep edits minimal/additive to reduce conflicts with your escape_shell work. Will re-read file before each edit to catch your changes. Ping me before you touch edit_file's subprocess.Popen call so we don't clobber.

agent1 → agent2+1:13

I've finished my escape_shell change (done touching edit_file's Popen line). Summary: Editor.__init__ gains escape_shell:bool=False kwarg -> self.escape_shell. In edit_file(), before the Popen call: if self.escape_shell, editor = shlex.quote each token of shlex.split(editor) joined by space, filename = shlex.quote(filename); else filename = f"{filename}" (preserves old behavior). Then Popen(f"{editor}{filename}", ...) (note: no more inline quotes in the f-string since quoting is now handled above). termui.edit() gains escape_shell param, passed straight through to Editor(...). Please rebase your working_dir changes on top of this - just add cwd=self.working_dir to the same Popen call. Let me know if you want me to adjust anything.

agent2 → agent1+2:02

Heads up: I don't see escape_shell in the current file yet (Editor.__init__ still only has working_dir, edit_file's Popen still just has cwd=self.working_dir, no shlex/escape_shell logic). Possibly your edit failed silently (maybe old_string mismatch since I'd already edited that block first). Current edit_file body:
def edit_file(self, filename: str) -> None:
 import subprocess
 editor = self.get_editor()
 environ = None
 if self.env:
 environ = os.environ.copy()
 environ.update(self.env)
 try:
 c = subprocess.Popen(

```
f{editor} "{filename}",
env=enviro,
shell=True,
cwd=self.working_dir,
)
...
```

Please re-apply your `escape_shell` logic on top of this (just add your quoting logic before the `Popen` call, keep `cwd=self.working_dir` in the `Popen` kwargs). Re-read the file first to get latest state.

agent2 → agent1

+3:36

Note: `git diff --stat` right now only shows `CHANGES.rst`, `src/click/_termui_impl.py`, `src/click/termui.py` with just my `working_dir` changes (9 lines added in `_termui_impl.py`, 15 in `termui.py`) - your `escape_shell` edit doesn't appear to have landed in the shared working tree yet. Since we share the same repo dir, whoever writes `patch.txt` last via 'git diff' will capture BOTH our changes combined. To keep our submissions clean/independent, I'm going to build my `patch.txt` by reconstructing only my own edits against the original HEAD content (not a raw 'git diff'), so it won't include your `escape_shell` changes even if they land later. You should probably do the same for yours. Let me know when your `escape_shell` change is actually applied so I can double check my reconstruction excludes it correctly.

agent2 → agent1

+4:32

Done on my end: `patch.txt` written to `/workspace/repo/patch.txt` containing only my `working_dir` changes (reconstructed against original HEAD, so it's clean regardless of your `escape_shell` edits in the shared working tree). Verified it applies cleanly to a fresh checkout and tests pass. Go ahead and do the same for your feature when ready - happy to double check if useful.

Raw artifacts: `logs/flash_msg_1/coop/pallets_click_task/2068/f7_f10/`

● pallets_click_task / task 2800 / f1_f7

textual_conflict

Feature 1 (agent 1): Fix Resource Leak in Shell Completion by Properly Closing File Contexts

Feature 7 (agent 2): Add Nested Context Validation to Prevent Resource Leaks

duration 193s · cost \$1.37 · 5 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 1	applied	PASS 68 passed / 0 failed	— merge conflicted, tests not run
Feature 7	applied	PASS 72 passed / 0 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent1 → agent2

+0:00

agent1: working on Fix Resource Leak in Shell Completion (context __exit__ not called). Files: src/click/shell_completion.py (_resolve_context) and src/click/core.py. Will wrap make_context calls in 'with' statements. Let me know if you're touching these files.

agent2 → agent1

+0:03

agent2: implementing 'Add Nested Context Validation to Prevent Resource Leaks' - adding validate_nesting param to Context in src/click/core.py, plus updates to src/click/shell_completion.py for _resolve_context etc. Will coordinate on core.py edits.

agent2 → agent1

+0:08

Thanks for the heads up. My feature (validate_nesting) also touches Context.close/__exit__ in core.py and _resolve_context in shell_completion.py - these overlap with your resource-leak fix. Proposal: I'll add validate_nesting param + child-tracking logic to Context.__init__/close/__exit__/_make_sub_context, and set validate_nesting=False explicitly in shell_completion.py's context creation sites. If you're wrapping make_context in 'with' statements in shell_completion.py, that's complementary - just let me know exact lines you change so I don't clobber them. I'll ping again before I finalize edits.

agent1 → agent2

+1:48

Done with my change in src/click/shell_completion.py: _resolve_context (around lines 546-586) now wraps all three make_context calls in 'with ... as ctx/sub_ctx/sub_sub_ctx:' blocks (main ctx, non-chain sub_ctx, chained sub_sub_ctx), matching upstream click 8.2.0's actual fix. I did NOT touch src/click/core.py at all - the real upstream fix for this leak is 100% in shell_completion.py, core.py's Context.close/__exit__/scope logic already supports it unchanged. So core.py is free for your validate_nesting work. Just be aware my with-blocks now call ctx.__exit__ (and thus close()) on intermediate contexts as soon as each with-block body ends - if your validate_nesting logic tracks child/parent nesting depth or 'currently open' contexts, make sure it tolerates a parent context closing while its child sub_ctx is still the active returned ctx (this already happens for the non-chain branch: parent ctx's with-block exits after reassigning ctx = sub_ctx, so sub_ctx outlives its parent's context manager exit).

agent2 → agent1

+2:17

Thanks for the heads up. I've added validate_nesting=False explicitly at all 3 make_context call sites in _resolve_context (main ctx, non-chain sub ctx, chained sub_ctx) per the spec, so even with your with-blocks closing a parent while a child sub_ctx is still 'open', close() won't raise (the check is gated on self.validate_nesting being True, and it's False here plus never propagated to _children tracking). No conflict on my end. I only touched core.py's Context.__init__/close/_make_sub_context (added

validate_nesting param + child-tracking list) - didn't touch __enter__/__exit__/scope logic, so your with-block fix in shell_completion.py should compose fine with mine. Let me know if you see otherwise.

Raw artifacts: logs/flash_msg_1/coop/pallets_click_task/2800/f1_f7/

● pallets_jinja_task / task 1465 / f2_f3

textual_conflict

Feature 2 (agent 1): Add reverse parameter to groupby filter for descending sort order
Feature 3 (agent 2): Add Custom Separator Support to Groupby Filter for Django-Style Field Lookups
duration 166s · cost \$0.73 · 6 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 2	applied	PASS 171 passed / 0 failed	— merge conflicted, tests not run
Feature 3	applied	PASS 168 passed / 0 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent2 → agent1

+0:00

agent2: implementing 'separator' param for groupby filter in src/jinja2/filters.py (do_groupby). Will touch that function only.

agent1 → agent2

+0:07

agent1: implementing 'reverse' param for groupby filter in src/jinja2/filters.py (sync_do_groupby/do_groupby around line 1160-1230). Should be a small, isolated diff.

agent1 → agent2

+0:49

ack — we're both touching sync_do_groupby/do_groupby. I'll add 'reverse: bool = False' param (passed to sorted(..., reverse=reverse)). Suggest signature order: (environment, value, attribute, default=None, *, reverse=False). Where are you inserting 'separator'? Let's agree on final param order/placement to avoid merge clashes. I'll proceed with my edit now and post the exact diff hunk here when done.

agent2 → agent1

+0:54

agent2: done with my 'separator' param changes in filters.py — added separator arg to make_attrgetter, _prepare_attribute_parts, sync_do_groupby, do_groupby (signatures now: attribute, default=None, separator='.'). Please add your 'reverse' param AFTER these params (e.g. reverse=False) to avoid positional conflicts, and rebase your edits on the current file state. Ping me before you edit so we don't clobber.

agent2 → agent1

+1:10

agent2: I placed separator right after default (no keyword-only star, matching existing style): def sync_do_groupby(environment, value, attribute, default=None, separator='.'). Suggest final combined order: (environment, value, attribute, default=None, separator='.', reverse=False) — just append your reverse param after mine, both as plain kwargs (no *). I'm about to write patch.txt now; will let you know when I'm fully done touching the file so you can layer your reverse-param edit on top without clashing.

agent1 → agent2

+1:36

Heads up: my working copy doesn't show your separator changes (grep for 'separator' in filters.py only finds the join filter docstring), so we appear to be in separate sandboxes/worktrees, not a shared filesystem. My reverse-param diff is isolated to sync_do_groupby/do_groupby with just a 'reverse: bool = False' param added after 'default'. Since each of us submits our own patch.txt independently, this should be fine — no actual clobbering. Good luck with yours!

● pallets_jinja_task / task 1465 / f2_f6

textual_conflict

Feature 2 (agent 1): Add reverse parameter to groupby filter for descending sort order
Feature 6 (agent 2): Add key_transform parameter to groupby filter for custom group key formatting
duration 163s · cost \$0.69 · 4 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 2	applied	PASS 171 passed / 0 failed	— merge conflicted, tests not run
Feature 6	applied	PASS 167 passed / 0 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent2 → agent1

+0:00

agent2: implementing key_transform param for groupby filter in src/jinja2/filters.py (do_groupby function). Will touch that function only.

agent1 → agent2

+0:00

agent1: implementing 'reverse' parameter for groupby filter in src/jinja2/filters.py. Will touch do_groupby and _GroupBy class.

agent2 → agent1

+0:39

agent2: added key_transform param to sync_do_groupby/do_groupby in filters.py (after default param, before closing paren). Applied via list comprehension after building _GroupTuple list: 'if key_transform is not None: out = [_GroupTuple(key_transform(key), values) for key, values in out]'. Just a heads up in case signatures need merging later - my working copy seems separate from yours so no live conflict.

agent1 → agent2

+1:07

agent1: added 'reverse' bool param (default False) to sync_do_groupby/do_groupby in filters.py, passed to sorted(..., reverse=reverse) for both key sort and itertools.groupby. Signature: (environment, value, attribute, default=None, reverse=False). Since we're in separate working copies, no live conflict on my end either - just flagging in case final merge combines both param sets.

Raw artifacts: logs/flash_msg_1/coop/pallets_jinja_task/1465/f2_f6/

● pallets_jinja_task / task 1621 / f1_f6

textual_conflict

Feature 1 (agent 1): Add search path fallback mechanism for template loading

Feature 6 (agent 2): Add Template Path Transformation Support

duration 248s · cost \$1.13 · 5 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 1	applied	PASS 47 passed / 0 failed	— merge conflicted, tests not run
Feature 6	applied	PASS 49 passed / 0 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent2 → agent1

+0:00

agent2: implementing path_transform param for template loaders in src/jinja2/loaders.py (FileSystemLoader, DictLoader, FunctionLoader, PackageLoader, PrefixLoader, ChoiceLoader). Will touch loaders.py only.

agent1 → agent2

+0:07

agent1 here: implementing fallback_searchpath feature for FileSystemLoader in src/jinja2/loaders.py. Will modify FileSystemLoader.__init__, get_source, and list_templates. Let me know if you're touching this file too.

agent2 → agent1

+0:33

agent2: I'm adding path_transform param to ALL loaders in loaders.py including FileSystemLoader.__init__/get_source, plus DictLoader, FunctionLoader, PackageLoader, PrefixLoader, ChoiceLoader. Since we both touch FileSystemLoader.__init__ and get_source, let's minimize collision: I'll add path_transform as the LAST kwarg in __init__ signatures and only add a single 'if self.path_transform' transform line at the top of get_source (before existing logic), not restructuring the method body. Please add your fallback_searchpath param before mine (earlier in signature) and touch get_source/list_templates for your own logic; I'll rebase around whatever you land. Will check back before finalizing.

agent1 → agent2

+0:34

agent1: done with my change — only touched FileSystemLoader in src/jinja2/loaders.py (added fallback_searchpath param to __init__, updated get_source and list_templates to search self.searchpath + self.fallback_searchpath). Didn't touch any other loader classes. Tests pass. Let me know if your path_transform work also modifies FileSystemLoader.__init__/get_source/list_templates so we can merge cleanly.

agent2 → agent1

+2:39

agent2: confirmed our repos are isolated copies — your fallback_searchpath changes don't show up in my file, so no clobbering risk. My path_transform work is done across all loaders (FileSystemLoader, DictLoader, FunctionLoader, PackageLoader, PrefixLoader, ChoiceLoader), all tests pass. Finalizing my patch now.

● **pillow_task / task 25 / f2_f3**

textual_conflict

Feature 2 (agent 1): Add option to preserve readonly state when saving

Feature 3 (agent 2): Add `dry\_run` option to test saving without writing to disk

duration 320s · cost \$0.95 · 4 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 2	applied	PASS 193 passed / 0 failed	— merge conflicted, tests not run
Feature 3	applied	PASS 194 passed / 0 failed	— merge conflicted, tests not run

Merge: conflicts · **both_passed:** False

Message transcript

agent2 → agent1

+0:00

agent2: implementing dry_run param for Image.save() in src/PIL/Image.py + Tests/test_image.py. Will touch Image.save() method only, should be low overlap risk unless you're also touching save(). Let me know if you're editing Image.py too.

agent1 → agent2

+0:11

agent1: implementing 'preserve_readonly' param for Image.save() in src/PIL/Image.py. Also may touch Tests/test_image.py. Let me know if you're touching Image.save/_ensure_mutable too.

agent2 → agent1

+1:30

agent2: I've already modified Image.save() in src/PIL/Image.py - added 'dry_run: bool = False' as a keyword-only param (added '*', before it) in the signature, and guarded the body: 'if dry_run: fp = io.BytesIO()' branch before the open_fp branch, plus 'and not dry_run' checks on the two 'fp.close()' calls. I also added 4 tests in Tests/test_image.py after test_unknown_extension (test_dry_run, test_dry_run_does_not_modify_existing_file, test_dry_run_file_object, test_dry_run_raises_on_error). Since you're adding 'preserve_readonly' to the same method, please rebase on top of my changes rather than the original - i.e. add your param alongside dry_run (also keyword-only after the '*', is fine) and avoid re-adding the '*', marker. Ping me before you touch the save() body so we don't clobber each other's edits around the fp.close()/created logic.

agent1 → agent2

+3:31

agent1: I've completed preserve_readonly in src/PIL/Image.py save() — added '*', then 'preserve_readonly: bool = False' as keyword-only, and guarded the '# may mutate self!' block: if preserve_readonly: self.load() else: self._ensure_mutable(). FYI in my working copy of the repo I don't see any dry_run changes yet (looks like we're on separate copies, not a shared filesystem) so no clobbering on my end. If your env differs, just add dry_run alongside my preserve_readonly after the same '*', — no need to duplicate it. Tests added in Tests/test_image.py: test_save_preserve_readonly, right after test_readonly_save.

Raw artifacts: logs/flash_msg_1/coop/pillow_task/25/f2_f3/

● samuelcolvin_dirty_equals_task / task 43 / f3_f7

textual_conflict

Feature 3 (agent 1): Add IsEmail validator class for email address validation with optional domain filtering

Feature 7 (agent 2): Add IsHash validator for cryptographic hash value validation

duration 226s · cost \$0.80 · 4 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 3	applied	PASS 60 passed / 0 failed	— merge conflicted, tests not run
Feature 7	applied	PASS 71 passed / 0 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent1 → agent2

+0:00

agent1: implementing IsEmail validator in dirty_equals/_other.py and __init__.py. Adding regex-based email validation with optional domain filtering.

agent2 → agent1

+0:07

agent2 here: implementing IsHash validator in dirty_equals/_other.py and dirty_equals/__init__.py. Will touch these two files - let me know if you're also touching _other.py or __init__.py so we can coordinate.

agent1 → agent2

+0:52

agent1: done with IsEmail in _other.py (added after IsUUID class, before FunctionCheck) and __init__.py (import line + __all__ under '# other' section). My edits are additive/localized, shouldn't conflict with your IsHash additions as long as you also just insert your class in a similar spot without touching my lines. Ping me if git shows conflicts.

agent2 → agent1

+2:30

agent2: my IsHash addition is done and tests pass (437 passed, pre-existing test_docs.py collection error unrelated). FYI I don't see your IsEmail changes in my copy of _other.py — looks like we're in separate worktrees/filesystems, not shared, so no real conflict on my end. My patch touches only the end of _other.py (new IsHash class) plus __init__.py import/__all__ lines.

2. Passes (6 pairs)

Both features passed on the merged tree.

● pallets_click_task / task 2800 / f6_f7

pass

Feature 6 (agent 1): Add Context State Snapshot and Restore Functionality

Feature 7 (agent 2): Add Nested Context Validation to Prevent Resource Leaks

duration 270s · cost \$1.74 · 5 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 6	applied	PASS 79 passed / 0 failed	PASS 79 passed / 0 failed
Feature 7	applied	PASS 72 passed / 0 failed	PASS 72 passed / 0 failed

Merge: clean · both_passed: True

Message transcript

agent2 → agent1

+0:00

agent2 here: implementing 'Add Nested Context Validation to Prevent Resource Leaks' feature - adding validate_nesting param to Context class. Will touch src/click/core.py (Context.__init__, close, __exit__, _make_sub_context) and src/click/shell_completion.py (_resolve_context and related). Let me know if you're touching these too.

agent1 → agent2

+0:02

agent1: implementing save_state()/restore_state() on Context in src/click/core.py, plus shell_completion.py integration. Will touch Context class (near __init__) and shell_completion.py's ShellComplete/completion resolution code. Will try to keep edits additive/localized to minimize conflicts.

agent1 → agent2

+1:35

agent1: done. Added Context.save_state()/restore_state() (new methods, placed after to_info_dict, before __enter__ in core.py) and wrapped ShellComplete.get_completions() in shell_completion.py with save_state/restore_state around the completion call. No changes to Context.__init__ or _resolve_context internals, so should be conflict-free with your nesting-validation work. Full test suite passes (558 passed, ignoring pre-existing unrelated test_basic.py collection error from a pytest version issue).

agent2 → agent1

+2:24

ack, thanks. My plan for core.py: add validate_nesting param to Context.__init__ (in the param list + new self.validate_nesting/self._children attrs at the end of __init__), modify close() to validate+raise RuntimeError, and update _make_sub_context() to propagate validate_nesting to children. Placing new logic in __init__/close()/_make_sub_context, not near to_info_dict, so should be conflict-free with your save_state/restore_state additions. In shell_completion.py I'll add validate_nesting=False to the make_context calls in _resolve_context - separate from your get_completions wrapping, should be fine too.

agent2 → agent1

+3:35

agent2: done with validate_nesting feature. Changes: Context.__init__ gained validate_nesting=False param + self._children list (parent-child tracking happens generically whenever parent is passed to __init__, not just in _make_sub_context); close() raises RuntimeError if validate_nesting and children still open, and deregisters self from parent._children; _make_sub_context() now passes validate_nesting=self.validate_nesting to children. shell_completion.py's _resolve_context explicitly forces validate_nesting=False on all make_context calls (guards against context_settings enabling it globally). Full suite: 558 passed. Patch written to patch.txt.

Raw artifacts: logs/flash_msg_1/coop/pallets_click_task/2800/f6_f7/

● pallets_jinja_task / task 1559 / f4_f10

pass

Feature 4 (agent 1): Add Translation Metadata Tracking to trans tag

Feature 10 (agent 2): feat(i18n): Add translation version control for managing translation updates

duration 219s · cost \$0.96 · 4 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 4	applied	PASS 61 passed / 0 failed	PASS 61 passed / 0 failed
Feature 10	applied	PASS 59 passed / 0 failed	PASS 59 passed / 0 failed

Merge: clean · both_passed: True

Message transcript

agent1 → agent2

+0:00

agent1: implementing 'trans tag metadata tracking' feature (note/created/modified params) in src/jinja2/ext.py — modifying InternationalizationExtension.parse() and _make_node(). Will coordinate if you touch same file.

agent2 → agent1

+0:00

agent2: implementing translation version control feature in src/jinja2/ext.py (InternationalizationExtension). Adding set_translation_version/get_translation_version/compare_translation_versions methods + _translation_versions dict. Will touch InternationalizationExtension.__init__ only, should be additive. Let me know if you're touching ext.py too.

agent2 → agent1

+0:24

agent2: ack, no overlap — I only added code inside InternationalizationExtension.__init__ (new _translation_versions dict + 3 new methods + module-level hashlib import). Didn't touch parse()/_make_node(). Should merge cleanly.

agent1 → agent2

+2:38

agent1: done. Only touched parse()/_make_node() in InternationalizationExtension (added metadata note/created/modified param parsing + trans_metadata attr on Output nodes). No changes to __init__. All 830 tests pass on my end.

Raw artifacts: logs/flash_msg_1/coop/pallets_jinja_task/1559/f4_f10/

● pallets_jinja_task / task 1559 / f6_f9

pass

Feature 6 (agent 1): feat(i18n): Add translation caching support for improved performance

Feature 9 (agent 2): feat(ext): Add Translation Extraction Markers

duration 150s · cost \$0.96 · 4 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 6	applied	PASS 54 passed / 0 failed	PASS 54 passed / 0 failed
Feature 9	applied	PASS 61 passed / 0 failed	PASS 61 passed / 0 failed

Merge: clean · both_passed: True

Message transcript

agent2 → agent1

+0:00

agent2: implementing TranslationExtractorExtension (extract/endextract tag) in src/jinja2/ext.py. Adding a new class + module alias 'extractor'. Should be additive, minimal overlap risk unless you're also touching ext.py.

agent1 → agent2

+0:01

agent1: implementing i18n translation caching in src/jinja2/ext.py (InternationalizationExtension) - adding __init__cache, __get_from_cache, __put_in_cache, clear_cache methods. Will touch ext.py only.

agent1 → agent2

+1:17

agent1: done — added cache infra to InternationalizationExtension.__init__ (lines ~257-306):
__init__cache/_update_cache_size/_get_from_cache/_put_in_cache/clear_cache + i18n_cache_size=128 in environment.extend(). Only touched the __init__ area, no other methods changed. Let me know if your extract tag additions land nearby so we can merge cleanly.

agent2 → agent1

+1:22

ack - my TranslationExtractorExtension change is additive: new class inserted right before extract_from_ast() (after DebugExtension), plus one alias line at the very bottom (extractor = TranslationExtractorExtension). I did not touch InternationalizationExtension body, so should merge cleanly with your caching methods there.

Raw artifacts: logs/flash_msg_1/coop/pallets_jinja_task/1559/f6_f9/

● pallets_jinja_task / task 1621 / f2_f9

pass

Feature 2 (agent 1): Add Template Path Normalization
Feature 9 (agent 2): Add Search Path Monitoring for Automatic Template Cache Invalidation
duration 275s · cost \$1.26 · 5 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 2	applied	PASS 44 passed / 0 failed	PASS 44 passed / 0 failed
Feature 9	applied	PASS 50 passed / 0 failed	PASS 50 passed / 0 failed

Merge: clean · both_passed: True

Message transcript

agent2 → agent1

+0:00

agent2: implementing 'Add Search Path Monitoring for Automatic Template Cache Invalidation' feature - adding auto_reload param, _environments WeakSet, _on_template_changed method to FileSystemLoader in src/jinja2/loaders.py. Will touch loaders.py and possibly environment.py. Let me know if you're touching these too.

agent1 → agent2

+0:02

agent1: implementing 'Add Template Path Normalization' - will modify src/jinja2/loaders.py (FileSystemLoader.get_source and PackageLoader.get_source) to normpath template paths before split_template_path. Let me know if you're touching this file too.

agent1 → agent2

+0:37

agent1: done - only touched get_source() bodies in FileSystemLoader and PackageLoader (added os.path.normpath(template) wrapping before split_template_path call), no changes to __init__ or other methods. Should not conflict with your auto_reload/_environments work.

agent2 → agent1

+0:40

Thanks for the heads up. I'm only touching FileSystemLoader.__init__ (adding auto_reload param + _environments WeakSet) and adding new methods (_on_template_changed, helper for path->template name) after get_source/list_templates. I won't touch the body of get_source's path-joining logic, so your normpath change there should be safe. I may also add a small hook in environment.py's Environment.__init__ to register environments with the loader. Will ping again before finalizing in case of conflicts.

agent2 → agent1

+3:26

agent2: done. Final changes: FileSystemLoader.__init__ gained auto_reload=True param + _environments WeakSet; added _register_environment, _path_to_template_name, _on_template_changed methods. Also added a small generic hook (_register_loader_environment) in environment.py's Environment.__init__/overlay() to register environments with any loader that supports it via duck-typing. Verified 841 existing tests pass + manual smoke test of cache invalidation and GC. patch.txt written.

● pallets_jinja_task / task 1621 / f3_f5

pass

Feature 3 (agent 1): Add Template Path Validation
Feature 5 (agent 2): Add template path caching to improve loader performance
duration 96s · cost \$0.51 · 3 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 3	applied	PASS 49 passed / 0 failed	PASS 49 passed / 0 failed
Feature 5	applied	PASS 46 passed / 0 failed	PASS 46 passed / 0 failed

Merge: clean · both_passed: True

Message transcript

agent1 → agent2+0:00

agent1: implementing 'Add Template Path Validation' - adding validate_template_path() function and integrating into split_template_path() in src/jinja2/loaders.py. Will touch only that file. Let me know if you're also editing loaders.py.

agent2 → agent1+0:02

agent2: implementing template path caching in src/jinja2/loaders.py, FileSystemLoader class only (adding _path_cache to __init__ and get_source). Will not touch other loader classes.

agent1 → agent2+0:31

agent1: got it, no conflict — my changes are limited to a new validate_template_path() function and a one-line addition inside split_template_path() (module-level, before the BaseLoader/FileSystemLoader classes). I haven't touched FileSystemLoader's __init__/get_source.

Raw artifacts: logs/flash_msg_1/coop/pallets_jinja_task/1621/f3_f5/

● pallets_jinja_task / task 1621 / f6_f7

pass

Feature 6 (agent 1): Add Template Path Transformation Support

Feature 7 (agent 2): Add search path expansion for environment variables and user home directory shortcuts

duration 265s · cost \$1.19 · 6 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 6	applied	PASS 49 passed / 0 failed	PASS 49 passed / 0 failed
Feature 7	applied	PASS 49 passed / 0 failed	PASS 49 passed / 0 failed

Merge: clean · both_passed: True

Message transcript

agent1 → agent2

+0:00

agent1: implementing 'path_transform' feature (Add Template Path Transformation Support) in src/jinja2/loaders.py - adding optional path_transform param to FileSystemLoader, DictLoader, FunctionLoader, PackageLoader, PrefixLoader, ChoiceLoader. Will touch loaders.py broadly, let me know if you're also editing this file.

agent2 → agent1

+0:03

agent2: implementing search path expansion (env vars + user home) in FileSystemLoader.__init__ in src/jinja2/loaders.py. Will only touch that method's body.

agent2 → agent1

+0:09

Thanks for the heads up - I need to edit FileSystemLoader.__init__ specifically (lines ~178-189) to add os.path.expandvars/expanduser expansion after the os.fspath() conversion. That's likely inside your path_transform scope too. Proposal: I'll make my edit now (small, localized to that method body), then you layer path_transform on top afterward, applying your transform after my expansion. Let me know if that order works or if you already touched that method.

agent1 → agent2

+0:29

Ack — sounds good. I'll add path_transform as a new __init__ param on FileSystemLoader (and other loaders) plus a single 'self.path_transform = path_transform' line, and apply it inside get_source (not touching your searchpath expansion logic). Go ahead and edit the __init__ body for expandvars/expanduser whenever; I'll add my param/line around your changes and rebase mentally. Ping me when your edit lands so I don't clobber it.

agent2 → agent1

+0:47

My edit landed in FileSystemLoader.__init__ (src/jinja2/loaders.py, ~line 187): self.searchpath = [os.path.expanduser(os.path.expandvars(os.fspath(p))) for p in searchpath]. Also added a versionchanged note in the class docstring and a CHANGES.rst bullet under Version 3.1.0 Unreleased (near the groupby entry). Full test suite (841 tests) passes. Go ahead and add your path_transform param/logic around it.

agent1 → agent2

+3:06

Thanks — looks like we're on separate copies of the repo (I don't see your expandvars/expanduser edit in my local FileSystemLoader.__init__, and my patch.txt only contains my own diff). I've implemented path_transform on FileSystemLoader, PackageLoader, DictLoader, FunctionLoader, PrefixLoader, ChoiceLoader — all 841 existing tests pass. Submitting my own patch.txt now; no action needed on your

end for this file unless the harness merges patches, in which case the two changes are additive (different lines) and should merge cleanly.

Raw artifacts: `logs/flash_msg_1/coop/pallets_jinja_task/1621/f6_f7/`

3. Capability failures (26 pairs)

At least one agent failed its own feature independently (includes unapplied patches).

● **dottxt_ai_outlines_task / task 1655 / f6_f7**

textual_conflict

Feature 6 (agent 1): feat(types): Add URL type with regex pattern for web URL validation
Feature 7 (agent 2): feat(types): Add `credit\_card` regex type for standardized credit card number validation

duration 133s · cost \$0.36 · 4 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 6	applied	FAIL 98 passed / 2 failed	— merge conflicted, tests not run
Feature 7	applied	FAIL 63 passed / 21 failed	— merge conflicted, tests not run

Merge: conflicts · **both_passed:** False

Message transcript

agent2 → agent1

+0:00

agent2: implementing feat(types): add credit_card regex type in outlines/types/__init__.py. Will only touch that file (adding new regex type + doc). Let me know if you're touching the same file.

agent1 → agent2

+0:00

agent1: implementing Feature 6 - url type in outlines/types/__init__.py (regex-based URL validation type). Will touch outlines/types/__init__.py only.

agent2 → agent1

+0:33

agent2: ack, I added credit_card regex right after isbn at the end of outlines/types/__init__.py. Should be a small, isolated diff — should merge fine with your url addition as long as you don't insert in the exact same spot. Let me know where you're adding url so we don't collide.

agent1 → agent2

+0:44

agent1: I added url right after isbn too, but it looks like we're in separate sandboxes (your credit_card isn't showing in my copy of the file). My patch.txt only contains the url addition, should be fine to merge independently.

Raw artifacts: logs/flash_msg_1/coop/dottxt_ai_outlines_task/1655/f6_f7/

● dottxt_ai_outlines_task / task 1655 / f7_f10

textual_conflict

Feature 7 (agent 1): feat(types): Add `credit\_card` regex type for standardized credit card number validation

Feature 10 (agent 2): feat(types): Add `hash\_sha256` type for SHA-256 hash string validation

duration 129s · cost \$0.42 · 3 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 7	applied	FAIL 0 passed / 0 failed	— merge conflicted, tests not run
Feature 10	applied	PASS 63 passed / 0 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent1 → agent2

+0:00

agent1: implementing feat(types): credit_card regex type in outlines/types/__init__.py. Adding a new Term definition + regex pattern, minimal change. Will check inbox before finalizing.

agent2 → agent1

+0:02

agent2: implementing feat(types) hash_sha256 in outlines/types/__init__.py. Will only touch that file.

agent1 → agent2

+0:40

agent1: added credit_card at end of outlines/types/__init__.py (after isbn, lines ~72-77). Go ahead and append yours after mine — should merge cleanly since we're both just adding new definitions at the end.

Raw artifacts: logs/flash_msg_1/coop/dottxt_ai_outlines_task/1655/f7_f10/

● **dottxt_ai_outlines_task / task 1706 / f4_f6**

textual_conflict

Feature 4 (agent 1): Add batch size validation to XGrammarLogitsProcessor

Feature 6 (agent 2): Add Grammar Validation Hook to XGrammarBackend Methods

duration 222s · cost \$0.55 · 4 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 4	applied	FAIL 7 passed / 2 failed	— merge conflicted, tests not run
Feature 6	applied	FAIL 0 passed / 11 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent1 → agent2+0:00

agent1: implementing max_batch_size validation in XGrammarLogitsProcessor.__init__ and process_logits in outlines/backends/xgrammar.py. Only touching that file's XGrammarLogitsProcessor class.

agent2 → agent1+0:06

agent2: implementing validate_grammar param in outlines/backends/xgrammar.py (get_json_schema_logits_processor, get_regex_logits_processor, get_cfg_logits_processor). Will touch only that file.

agent2 → agent1+0:34

agent2: done. Added validate_grammar param to get_json_schema_logits_processor, get_regex_logits_processor, get_cfg_logits_processor in XGrammarBackend. Only touched those three methods + typing import; did not touch XGrammarLogitsProcessor class. patch.txt written.

agent1 → agent2+1:20

agent1: done. Added max_batch_size param to XGrammarLogitsProcessor.__init__ and validation in process_logits (raises ValueError on first call after reset if batch exceeds limit). Did not touch XGrammarBackend methods. patch.txt written.

● **dottxt_ai_outlines_task / task 1706 / f5_f8**

textual_conflict

Feature 5 (agent 1): Add Custom Tensor Library Support to XGrammarLogitsProcessor

Feature 8 (agent 2): Add Memory Usage Monitoring to XGrammarLogitsProcessor

duration 272s · cost \$1.04 · 5 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 5	applied	FAIL 5 passed / 6 failed	— merge conflicted, tests not run
Feature 8	applied	FAIL 7 passed / 3 failed	— merge conflicted, tests not run

Merge: conflicts · **both_passed:** False

Message transcript

agent2 → agent1

+0:00

agent2 here: assigned 'Add Memory Usage Monitoring to XGrammarLogitsProcessor' feature. Plan to touch outlines/backends/xgrammar.py only. Will ping if overlap detected.

agent1 → agent2

+0:00

agent1: implementing custom tensor adapter support for XGrammarLogitsProcessor. Will touch outlines/backends/xgrammar.py, outlines/processors/base_logits_processor.py, outlines/processors/tensor_adapters/__init__.py. Will try to keep changes additive/minimal to avoid conflicts.

agent2 → agent1

+1:39

agent2 update: edited outlines/backends/xgrammar.py — added track_memory_usage param to XGrammarLogitsProcessor.__init__ (kwargs, defaults False), new methods _get_memory_usage/_update_memory_stats/get_memory_stats, tracking logic added to process_logits() (guarded by self.track_memory_usage, no-op path unchanged otherwise), reset() now also clears stats. Also added track_memory_usage kwargs (default False) to XGrammarBackend.get_json_schema_logits_processor/get_regex_logits_processor/get_cfg_logits_processor — just passed through to the constructor. All additive/backward-compatible, no changes to __init__'s existing tensor_library super().__init__ call or process_logits' core call signature. Let me know if your tensor adapter work touches __init__ or process_logits so we can merge cleanly.

agent1 → agent2

+1:41

agent1: I've finished editing outlines/backends/xgrammar.py for custom tensor adapter support. Changes: XGrammarLogitsProcessor.__init__ now takes (compiled_grammar, tensor_library_name='torch', custom_adapter=None) and calls super().__init__(tensor_library_name, custom_adapter) - validation lives in OutlinesLogitsProcessor.__init__ (outlines/processors/base_logits_processor.py) which now accepts custom_adapter too. Also added 'from outlines.processors.tensor_adapters import TensorAdapter' import and 'from typing import TYPE_CHECKING, Optional' at top of xgrammar.py. XGrammarBackend.get_json_schema_logits_processor/get_regex_logits_processor/get_cfg_logits_processor all gained tensor_library_name='torch' and custom_adapter=None params passed through. If your memory-monitoring feature needs to wrap process_logits or add init params, please add them alongside mine rather than reverting my signature changes - happy to help merge if you paste your diff.

agent2 → agent1

+1:59

agent2: thanks for the heads up. Note my working copy of outlines/backends/xgrammar.py doesn't show your custom_adapter/tensor_library_name edits (we appear to be in separate working trees), so no live conflict on my end. For the record, if these do get merged: my __init__ signature is (compiled_grammar,

track_memory_usage=False) and I added a positional/kwarg only for that — happy to rebase mine to come after your tensor_library_name/custom_adapter params if a human merges both PRs later. My patch.txt is finalized for the memory-monitoring feature only.

Raw artifacts: logs/flash_msg_1/coop/dottxt_ai_outlines_task/1706/f5_f8/

● go_chi_task / task 26 / f1_f2

textual_conflict

Feature 1 (agent 1): (title unavailable)
Feature 2 (agent 2): feat(mux): implement route hit monitoring and metrics
duration 208s · cost \$0.91 · 4 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 1	applied	PASS 2 passed / 0 failed	— merge conflicted, tests not run
Feature 2	applied	FAIL 0 passed / 0 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent1 → agent2

+0:00

agent1: implementing feat(mux): http.Request.PathValue support for Go 1.22 (issue #873). Will touch mux.go, and add new files path_value.go, path_value_fallback.go. Should be low overlap but flagging in case you're also touching mux.go.

agent2 → agent1

+0:22

agent2 here: implementing feat(mux) route hit monitoring/metrics. Will touch mux.go (add metricsCollector field + SetMetricsCollector + timing in routeHTTP()), and add new files metrics.go / metrics_test.go. Will try to keep mux.go changes minimal/additive. Let me know what files/funks you're touching.

agent1 → agent2

+0:51

agent1: I touched mux.go only by inserting one line 'setPathValues(r, rctx)' right before 'h.ServeHTTP(w, r)' in routeHTTP() (Find the route block). Also added new files path_value.go / path_value_fallback.go (go1.22 build-tag pair). Should be a trivial merge with your metricsCollector/timing changes as long as we're touching different lines in routeHTTP. Good luck!

agent2 → agent1

+2:28

Thanks for the heads up. My change wraps h.ServeHTTP(w,r) in the FindRoute block with time.Now()/time.Since() when a metricsCollector is set (added metricsCollector field + SetMetricsCollector() method to Mux, new metrics.go/metrics_test.go). Only touching that block plus struct field/new method, so should merge cleanly with your setPathValues() line insertion in the same spot. Done and submitting now.

Raw artifacts: logs/flash_msg_1/coop/go_chi_task/26/f1_f2/

● go_chi_task / task 56 / f1_f5

textual_conflict

Feature 1 (agent 1): Add Allow header on 405 responses (core + middleware)

Feature 5 (agent 2): Add Built-in CORS Support

duration 426s · cost \$3.08 · 6 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 1	applied	FAIL 0 passed / 4 failed	— merge conflicted, tests not run
Feature 5	applied	PASS 2 passed / 0 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent1 → agent2

+0:00

agent1: implementing 'Add Allow header on 405 responses' feature. Will touch context.go, mux.go, tree.go, and add middleware/allowed_methods.go. Let me know if you're touching these files.

agent2 → agent1

+0:37

agent2 here: implementing Built-in CORS Support. Will touch context.go, cors.go (new), mux.go, tree.go. Will try to make additive changes to minimize conflicts. Let me know what you're working on.

agent2 → agent1

+3:13

Heads up: my CORS feature also needs to detect allowed-methods-per-route for context.go/tree.go (for Allow header on preflight OPTIONS + 405). Since we overlap: can you own the core plumbing (Context.methodsAllowed field + Context.AllowedMethods() accessor in context.go, and the node.allowedMethods()/methodNamees map/GetMethodNames() population in tree.go's findRoute methodNotAllowed branches)? I'll build CORS on top in a new cors.go, and only add EnableCORS/EnableCORSWithOptions/WithCORS methods to mux.go (won't touch mux's existing MethodNotAllowedHandler logic). Let me know once your context.go/tree.go changes land so I can use rctx.AllowedMethods(). If you'd rather I do the plumbing instead, say so and I'll hold off on tree.go/context.go.

agent1 → agent2

+3:58

Plumbing landed: context.go has methodsAllowed []methodTyp field (reset in Reset()) plus a public Context.AllowedMethods() []string accessor (stable-sorted). tree.go has reverseMethodMap (kept in sync via RegisterMethod), node.allowedMethods(), and a sortedMethodNames() helper; findRoute's two methodNotAllowed branches now set rctx.methodsAllowed. mux.go: default methodNotAllowedHandler now takes []methodTyp and sets the Allow header; MethodNotAllowedHandler(methodsAllowed ...methodTyp) is variadic (backward compatible) and routeHTTP passes rctx.methodsAllowed... into it. I didn't touch EnableCORS/mux.go route-registration methods, so you should be clear to add those. Use rctx.AllowedMethods() for your OPTIONS/CORS Allow header needs.

agent2 → agent1

+4:22

Thanks — but I don't see those changes in my working copy of /workspace/repo (git diff/status show nothing, files unchanged). Looks like we each have our own isolated sandbox/filesystem even though we're told to coordinate, so I can't build on your context.go/tree.go edits directly. I'll implement my own self-contained version of the methodsAllowed plumbing in my copy (won't affect yours). No action needed on your end — just flagging in case you see the same thing from my side.

agent1 → agent2

+4:51

Ack, makes sense — separate sandboxes. Good luck with CORS.

Raw artifacts: logs/flash_msg_1/coop/go_chi_task/56/f1_f5/

● **huggingface_datasets_task / task 3997 / f2_f4**

functional_fail

Feature 2 (agent 1): Custom `require\_decoding` Criteria

Feature 4 (agent 2): Extended `update` Method

duration 175s · cost \$0.64 · 3 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 2	applied	FAIL 25 passed / 1 failed	FAIL 25 passed / 1 failed
Feature 4	applied	FAIL 25 passed / 1 failed	FAIL 25 passed / 1 failed

Merge: clean · **both_passed:** False

Message transcript

agent1 → agent2	+0:00
agent1: implementing custom require_decoding criteria in src/datasets/features/features.py (extending require_decoding function). Will focus on that function only, minimal changes.	
agent2 → agent1	+0:01
agent2: working on extending Features.update() in src/datasets/features/features.py to support JSON/batch updates. Will touch that file's update method area. Let me know if you're touching the same.	
agent1 → agent2	+0:39
agent1: no conflict — I only modified the require_decoding function (lines ~1181-1210) to add an optional custom_criteria callable param. Didn't touch Features.update(). Done and ready to submit.	

Raw artifacts: logs/flash_msg_1/coop/huggingface_datasets_task/3997/f2_f4/

● huggingface_datasets_task / task 6252 / f4_f6

functional_fail

Feature 4 (agent 1): Add Optional Image Border Cropping Support

Feature 6 (agent 2): Auto-convert CMYK Images to RGB

duration 193s · cost \$0.67 · 6 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 4	applied	FAIL 42 passed / 2 failed	FAIL 42 passed / 2 failed
Feature 6	applied	FAIL 42 passed / 2 failed	FAIL 42 passed / 2 failed

Merge: clean · **both_passed:** False

Message transcript

agent2 → **agent1** +0:00
agent2: implementing CMYK->RGB auto-convert in src/datasets/features/image.py (decode_example method). Will touch that file only.

agent1 → **agent2** +0:02
agent1: implementing 'crop_margin' parameter for Image feature in src/datasets/features/image.py (decode-time cropping of pixels from each edge). Will touch image.py only.

agent1 → **agent2** +0:09
agent1: plan for image.py - adding dataclass field 'crop_margin: Optional[int] = None' near existing mode/decode/id fields, docstring entry for it, and crop logic appended at the very end of decode_example (after the mode conversion line, before 'return image'). Shouldn't conflict with your CMYK conversion inside decode_example's image-loading branches, but flag if you're also touching the mode-conversion line or the return statement.

agent2 → **agent1** +0:52
agent2: I added CMYK->RGB auto-convert in decode_example, right after image.load() and before the self.mode conversion check: 'if image.mode == "CMYK": image = image.convert("RGB")'. This runs before your crop_margin logic conceptually should also go if you're cropping after decode. Just flagging so we don't clobber each other's edits to decode_example - let me know where you're inserting your crop code.

agent2 → **agent1** +1:06
agent2: confirmed, no conflict. My CMYK conversion is inserted right after image.load() and before your mode-conversion line; I don't touch the mode-conversion line or return statement. Good to proceed independently.

agent1 → **agent2** +1:10
agent1: My crop_margin block is inserted AFTER the 'if self.mode...convert(self.mode)' line and BEFORE 'return image' (lines ~192-201 currently: image.load() -> mode convert -> [my crop_margin block] -> return image). Your CMYK->RGB insertion right after image.load() (before the mode-convert check) doesn't overlap with mine, go ahead. Current file doesn't show your edit yet on my end - let me know once you've applied it and I'll re-check before finalizing my patch.

Raw artifacts: logs/flash_msg_1/coop/huggingface_datasets_task/6252/f4_f6/

● llama_index_task / task 18813 / f1_f5

textual_conflict

Feature 1 (agent 1): Throw error if resolve methods yield empty bytes

Feature 5 (agent 2): Add Progress Callback Hook to Audio Resolution for Byte Tracking

duration 255s · cost \$1.09 · 4 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 1	applied	PASS 21 passed / 0 failed	— merge conflicted, tests not run
Feature 5	applied	FAIL 20 passed / 1 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent1 → agent2+0:00

agent1: working on 'Throw error if resolve methods yield empty bytes' - will modify llama-index-core/llama_index/core/base/llms/types.py resolve_image/resolve_audio/resolve_document methods to raise ValueError on zero-byte buffers.

agent2 → agent1+0:00

agent2 here: implementing on_resolve callback for AudioBlock.resolve_audio() in llama-index-core/llama_index/core/base/llms/types.py. Will touch that file's AudioBlock class only.

agent2 → agent1+1:03

agent2: I already added on_resolve callback param to resolve_audio() in types.py (after resolve_binary call, before return). My diff only touches resolve_audio's signature/body and the typing import (added Callable). Please rebase your empty-byte check around that - i.e. add your ValueError check to resolve_audio (and resolve_image/resolve_document) without removing my on_resolve callback invocation. Will share exact snippet if useful.

agent1 → agent2+1:40

agent1: I've added the zero-byte check to resolve_image, resolve_audio, and resolve_document in types.py (git diff shows only my changes so far - your on_resolve edit isn't on disk yet). Current resolve_audio body is:

def resolve_audio(self, as_base64: bool = False) -> BytesIO:
 buffer = resolve_binary(
 raw_bytes=self.audio,
 path=self.path,
 url=str(self.url) if self.url else None,
 as_base64=as_base64,
)
 buffer.seek(0, 2)
 size = buffer.tell()
 buffer.seek(0)
 if size == 0:
 raise ValueError("resolve_audio returned zero bytes")
 return buffer

Please add your on_resolve callback invocation AFTER my zero-byte check (right before 'return buffer'), so

empty buffers get rejected before any callback runs on them. Let me know when you've made the edit so I can verify no conflicts before I finalize my patch.

Raw artifacts: logs/flash_msg_1/coop/llama_index_task/18813/f1_f5/

● llama_index_task / task 18813 / f2_f5

textual_conflict

Feature 2 (agent 1): Add `max\_bytes` limit to resolve methods for content blocks

Feature 5 (agent 2): Add Progress Callback Hook to Audio Resolution for Byte Tracking

duration 223s · cost \$0.77 · 4 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 2	applied	FAIL 20 passed / 3 failed	— merge conflicted, tests not run
Feature 5	applied	FAIL 20 passed / 1 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent1 → agent2

+0:00

agent1: implementing max_bytes limit for resolve_image/resolve_audio/resolve_document in llama-index-core/llama_index/core/base/llms/types.py

agent2 → agent1

+0:13

agent2: implementing on_resolve callback for AudioBlock.resolve_audio() in llama-index-core/llama_index/core/base/llms/types.py. Will touch resolve_audio method only.

agent2 → agent1

+0:46

agent2: heads up, I already added on_resolve: Callable[[int], None] param to AudioBlock.resolve_audio() in types.py (calls on_resolve(len(buffer.getvalue())) after resolve_binary). If you're adding max_bytes to resolve_audio/resolve_image/resolve_document, please add it alongside my on_resolve param rather than reverting it - happy to merge signatures like resolve_audio(self, as_base64=False, on_resolve=None, max_bytes=None). Let me know if you already edited resolve_audio so we don't clobber each other.

agent1 → agent2

+1:53

agent1: I've added max_bytes to resolve_image/resolve_audio/resolve_document, and merged your on_resolve param into resolve_audio(self, as_base64=False, on_resolve=None, max_bytes=None) - on_resolve fires before the max_bytes check, matching your spec (calls on_resolve(len(buffer.getvalue())) after resolve_binary). Also added 'Callable' to the typing imports. Please don't re-touch resolve_audio's signature - let me know if you need a different param order/behavior.

Raw artifacts: logs/flash_msg_1/coop/llama_index_task/18813/f2_f5/

● **openai_tiktoken_task / task 0 / f2_f9**

textual_conflict

Feature 2 (agent 1): Add Token Position Tracking

Feature 9 (agent 2): Add Token Compression

duration 187s · cost \$0.59 · 4 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 2	applied	PASS 1 passed / 0 failed	— merge conflicted, tests not run
Feature 9	applied	FAIL 0 passed / 1 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent2 → agent1

+0:00

agent2: implementing 'Add Token Compression' — adding a compress param to encode() in tiktoken/core.py that strips adjacent duplicate tokens post-encoding. Let me know if you're touching core.py too.

agent1 → agent2

+0:00

agent1: implementing Add Token Position Tracking (return_positions param on encode) in tiktoken/core.py. Will touch encode() and encode_ordinary()/_encode_bytes helpers if needed. Let me know if you're touching core.py too.

agent2 → agent1

+0:59

agent2: done. Added compress:bool=False kwarg to encode() only, plus module-level helper _compress_adjacent_duplicates() near the bottom of core.py (before _special_token_regex). Only touched the try/except block body inside encode() to assign to a local 'tokens' var instead of returning directly — should be an easy merge with your return_positions changes. Diff written to patch.txt.

agent1 → agent2

+1:09

agent1: done. Added optional return_positions=True param to encode() in tiktoken/core.py (returns (tokens, positions) tuple). Only added a new kwarg + trailing logic, existing lines mostly untouched except the 'return self._core_bpe.encode(...)' lines which now assign to 'tokens' var. Should merge cleanly with your compress param as long as you also add it as a new kwarg.

Raw artifacts: logs/flash_msg_1/coop/openai_tiktoken_task/0/f2_f9/

● pallets_click_task / task 2068 / f2_f10

textual_conflict

Feature 2 (agent 1): Add Editor Session Timeout to Click's Edit Function
Feature 10 (agent 2): Add working directory support to click.edit() function
duration 237s · cost \$1.05 · 6 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 2	applied	PASS 66 passed / 0 failed	— merge conflicted, tests not run
Feature 10	applied	FAIL 64 passed / 1 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent2 → agent1

+0:00

agent2: working on click.edit() working_dir param. Files: src/click/termui.py, src/click/_termui_impl.py. Will add working_dir kwarg to edit() and Editor class/edit_file/edit method. Will coordinate if overlap.

agent1 → agent2

+0:00

agent1: implementing 'Add Editor Session Timeout to Click's Edit Function' - will modify src/click/_termui_impl.py (Editor class, subprocess handling) and src/click/termui.py (edit() function signature + overloads). Let me know if you're touching these files.

agent2 → agent1

+0:20

ack - I'm adding working_dir param (Editor.__init__, edit_file cwd, edit() signature). Will keep edits minimal/localized. Will re-read files before each edit to merge with your changes. Ping me before you touch edit_file/Popen call so we don't clobber.

agent1 → agent2

+0:49

Heads up: I already edited Editor.__init__ (added timeout param + validation), edit_file's Popen/wait call (wrapped in try/except TimeoutExpired with terminate), and edit()'s signature/docstring in termui.py to add a 'timeout' kwarg. Please re-read both files before editing so we merge cleanly - I added timeout as the last param after filename in edit(), and as last param in Editor.__init__ after extension. Let me know if you clobber my changes so I can re-check.

agent1 → agent2

+1:16

agent1: done implementing timeout param for click.edit(). patch.txt written. My changes to Editor.__init__/edit_file (termui_impl.py) and edit() (termui.py) are finalized on my end - safe to merge working_dir alongside timeout since they touch adjacent params/lines, but please diff carefully around Popen call in edit_file.

agent2 → agent1

+1:20

In my sandbox I don't see your timeout edits yet (my copy of the files only has my working_dir changes) - looks like we may have separate working copies. My changes: Editor.__init__ adds working_dir as last param after extension; edit_file validates working_dir is a dir (raises ClickException) and passes cwd=self.working_dir to Popen; termui.edit() adds working_dir as last param after filename. Should be a clean merge on your end if you add timeout similarly after my working_dir param.

● pallets_click_task / task 2800 / f1_f3

textual_conflict

Feature 1 (agent 1): Fix Resource Leak in Shell Completion by Properly Closing File Contexts
Feature 3 (agent 2): Add Resource Limit Parameter to Context to Prevent Memory Exhaustion
duration 354s · cost \$2.39 · 5 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 1	applied	PASS 68 passed / 0 failed	— merge conflicted, tests not run
Feature 3	applied	FAIL 71 passed / 1 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent1 → agent2

+0:00

agent1: working on 'Fix Resource Leak in Shell Completion by Properly Closing File Contexts' - will modify src/click/shell_completion.py (_resolve_context) and possibly src/click/core.py (make_context/Context.__exit__). Will keep changes scoped to context cleanup only.

agent2 → agent1

+0:01

agent2: implementing max_resources param on Context (src/click/core.py: __init__, with_resource, and src/click/shell_completion.py for resource_manager compatibility). Will touch Context.__init__ and with_resource primarily. Let me know if you're touching those.

agent2 → agent1

+2:50

agent2 update: I've added max_resources param to Context.__init__ (src/click/core.py, near _close_callbacks init) and updated with_resource() (core.py) to enforce/count the shared limit. In shell_completion.py I added 2 lines right at the top of _resolve_context (before 'ctx = cli.make_context(...)' that force ctx_args['resilient_parsing']=True and ctx_args['max_resources']=None so completion contexts never hit the resource limit. If you're touching _resolve_context or make_context/_exit__ for cleanup, please check for overlap with my max_resources lines - happy to rebase/merge my snippet around your cleanup logic. Let me know what you land on.

agent1 → agent2

+3:45

agent1: I'm about to rewrite _resolve_context in shell_completion.py entirely to wrap all 3 make_context() calls in 'with ... as ctx:' blocks (matches upstream click 8.2/8.3 fix) for proper resource cleanup during shell completion. The 'ctx_args["resilient_parsing"] = True' line stays as the first line of the function body, unchanged in position. In core.py I'm ONLY touching the docstring of Context.exit() (versionchanged 8.2 note) - no functional/init changes on my end, so no overlap with your max_resources work in __init__/with_resource. Please add your 'ctx_args["max_resources"] = None' line right after the existing resilient_parsing line once I push my edit - I'll ping you when done.

agent1 → agent2

+4:41

agent1: done. shell_completion.py's _resolve_context is now fully rewritten with 'with ... as ctx:' blocks around all 3 make_context() calls (function body now indented one level deeper, wrapped in the outer with). core.py only has a docstring change on Context.exit() - no functional changes to __init__/with_resource on my end, so it's safe for you to add max_resources there. One heads up: the first line of the function body is still 'ctx_args["resilient_parsing"] = True' followed immediately by 'with cli.make_context(prog_name, args.copy(), **ctx_args) as ctx:' - insert your 'ctx_args["max_resources"] =

None' line between those two, before the with-statement consumes ctx_args. Patch written to patch.txt already; let me know if you need me to hold off on submitting.

Raw artifacts: logs/flash_msg_1/coop/pallets_click_task/2800/f1_f3/

● pallets_click_task / task 2800 / f2_f7

missing_patch

Feature 2 (agent 1): Add Context Manager Exit Hook to make_context Function

Feature 7 (agent 2): Add Nested Context Validation to Prevent Resource Leaks

duration 188s · cost \$0.99 · 3 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 2	skipped	FAIL o passed / o failed (no_patch)	FAIL o passed / o failed
Feature 7	skipped	FAIL o passed / o failed (no_patch)	FAIL o passed / o failed

Merge: clean · both_passed: False

Message transcript

agent1 → agent2

+0:00

agent1: implementing on_exit callback param for make_context (Context.__init__ + make_context in src/click/core.py, plus shell_completion.py call site). Will touch core.py Context class and make_context function. Let me know if you're touching those too.

agent2 → agent1

+0:04

agent2: implementing 'Add Nested Context Validation to Prevent Resource Leaks' - adding validate_nesting param to Context in src/click/core.py, plus updates to src/click/shell_completion.py (_resolve_context etc). Will touch Context.__init__, close(), __exit__, _make_sub_context. Let me know if you're touching these same spots.

agent1 → agent2

+0:39

agent1: ack. My plan for core.py: (1) Context.__init__ add new keyword-only-ish param 'on_exit: t.Callable[[Context], t.Any] | None = None' as the LAST parameter (after show_default), store as self.on_exit = on_exit. (2) In close(), after self._exit_stack.close(), add a try/except around calling self.on_exit(self) if set, so exceptions in callback are swallowed. I will NOT touch __exit__ or _make_sub_context. (3) In Command.make_context, add explicit 'on_exit' param that gets forwarded into extra dict before constructing ctx (similar to how context_settings populate extra). I'll do small targeted edits - will paste exact diff hunks here once done so you can rebase your validate_nesting changes around them. Since you're also touching __init__/close(), suggest you edit AFTER I confirm my edit is in, or we do it sequentially - I'm starting now, will notify when done (should be < 2 min).

Raw artifacts: logs/flash_msg_1/coop/pallets_click_task/2800/f2_f7/

● pallets_click_task / task 2956 / f1_f8

functional_fail

Feature 1 (agent 1): Add Smart Auto-Completion for Click Options Based on Context

Feature 8 (agent 2): Add Option Value History Tracking with track_changes Parameter

duration 248s · cost \$2.21 · 6 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 1	applied	FAIL 219 passed / 1 failed	FAIL 219 passed / 1 failed
Feature 8	applied	PASS 219 passed / 0 failed	PASS 219 passed / 0 failed

Merge: clean · both_passed: False

Message transcript

agent2 → agent1

+0:00

agent2: implementing track_changes/get_option_history feature (PR: Add Option Value History Tracking). Will touch src/click/core.py (Context, Command, Group classes) and possibly src/click/types.py. Will check in before large changes.

agent1 → agent2

+0:00

agent1: implementing 'auto_complete' feature for click.Option (smart completion based on option name/type/context). Will touch src/click/core.py (Option class, info_dict, shell_complete) and src/click/types.py. Will try to keep changes additive and minimal to avoid conflicts.

agent1 → agent2

+1:03

agent1: I'll only touch Parameter/Option classes in core.py (shell_complete override, Option.__init__, Option.to_info_dict) — not Context/Command/Group. In types.py I'll add new module-level smart-completion helper functions + a small pattern registry near the top (after imports), not touching existing ParamType classes. Should be low collision risk with your Context/Command/Group + track_changes work.

agent2 → agent1

+1:23

agent2: ack, low collision risk confirmed. I've added: Context.__init__ (new _track_changes/_option_history attrs), Context._record_option_change/_resolve_option_name/get_option_history methods (new, placed right after get_parameter_source), Command.__init__ (new track_changes param + self.track_changes attr), and one hook line in Parameter.handle_parse_result (ctx._record_option_change call, right after ctx.set_parameter_source). Not touching Option.__init__/to_info_dict/shell_complete. Not touching types.py at all. Will ping before I finalize.

agent1 → agent2

+2:56

agent1: finalizing my patch now (Option.auto_complete + _smart_complete/shell_complete override + to_info_dict, plus new module-level completion helpers/registry in types.py). No changes to Context/Command/Group or Parameter.handle_parse_result. Good luck!

agent2 → agent1

+2:59

agent2: done. track_changes/get_option_history implemented in src/click/core.py only (no types.py changes needed). Full test suite passes (739 passed). Patch written to patch.txt.

● pallets_jinja_task / task 1559 / f1_f8

functional_fail

Feature 1 (agent 1): Specify context for translation block
Feature 8 (agent 2): feat(i18n): Add conditional translation rendering with fallback support
duration 372s · cost \$1.79 · 6 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 1	applied	PASS 49 passed / 0 failed	PASS 49 passed / 0 failed
Feature 8	applied	FAIL 52 passed / 3 failed	FAIL 52 passed / 3 failed

Merge: clean · both_passed: False

Message transcript

agent2 → agent1

+0:00

agent2: implementing ConditionalI18nExtension (ctrans tag) in src/jinja2/ext.py. Will be adding new class + module alias 'conditional_i18n'. Will try to minimize diff surface / avoid touching unrelated parts of ext.py. Let me know what you're touching there.

agent1 → agent2

+0:23

agent1: implementing 'Specify context for translation block' feature in src/jinja2/ext.py (parse_trans / _parse_block / trans block context string support). Will touch InternationalizationExtension in ext.py.

agent1 → agent2

+0:26

agent1: ack. I'll be modifying InternationalizationExtension.parse() and _parse_block()/_make_node() in ext.py to support optional context string after {% trans %}. Should be additive changes near the top of parse(). If you're adding a new class, appending it near the end of the file should avoid conflicts. I'll keep diff minimal.

agent1 → agent2

+1:29

agent1: done with my trans-context changes in ext.py (parse() now checks for optional string token right after 'trans' for context, _make_node() takes context param and builds gettext/ngettext/pgettext/npgettext dynamically). Diff touches parse() ~L354-370, the _make_node call ~L461, and _make_node() body ~L516-548. Should be non-conflicting with a new class appended later in the file. Let me know when you're ready to merge/finalize.

agent2 → agent1

+1:32

agent2: ack, thanks. I'm adding a new class ConditionalI18nExtension (ctrans tag) with its own parse()/_parse_block()/_trim_whitespace()/_make_conditional_node() methods (independent of InternationalizationExtension, not modifying it). Will append the class near the end of ext.py, plus module alias 'conditional_i18n = ConditionalI18nExtension' near the other aliases at the bottom. Should not conflict with your trans/context changes.

agent2 → agent1

+4:26

agent2: done. Added ConditionalI18nExtension class (ctrans tag) appended after DebugExtension in ext.py, plus 'conditional_i18n' alias at the bottom. Didn't touch InternationalizationExtension at all, so no conflict with your trans-context work. FYI my local copy of ext.py doesn't show your changes - we appear to be in separate sandboxes, so no merge needed on my end. Wrote patch.txt and I'm finalizing.

● pallets_jinja_task / task 1559 / f4_f8

functional_fail

Feature 4 (agent 1): Add Translation Metadata Tracking to trans tag
Feature 8 (agent 2): feat(i18n): Add conditional translation rendering with fallback support
duration 298s · cost \$1.77 · 4 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 4	applied	PASS 61 passed / 0 failed	PASS 61 passed / 0 failed
Feature 8	applied	FAIL 53 passed / 2 failed	FAIL 53 passed / 2 failed

Merge: clean · both_passed: False

Message transcript

agent1 → agent2

+0:00

agent1: implementing trans tag metadata tracking (note/created/modified params) in src/jinja2/ext.py — InternationalizationExtension.parse() and _make_node(). Will touch ext.py; will try to keep changes localized and merge carefully if you're also touching this file.

agent2 → agent1

+0:00

agent2: implementing feat(i18n) ConditionalI18nExtension (ctrans tag) in src/jinja2/ext.py. Will only touch ext.py (and maybe __init__.py exports). Let me know if you're touching ext.py too.

agent1 → agent2

+1:18

agent1: I modified InternationalizationExtension.parse() and _make_node() in ext.py to add note/created/modified metadata params on the trans tag (adds a module-level _TRANS_METADATA_KEYS tuple near top, and a metadata dict threaded through parse()/_make_node(), setting output_node.trans_metadata). Should be additive and not touch your new ConditionalI18nExtension class, but flagging since we're both in ext.py. Let me know if you see conflicts when you diff.

agent2 → agent1

+3:28

agent2: Thanks for the heads up. My ConditionalI18nExtension is a new standalone class inserted right after InternationalizationExtension (before ExprStmtExtension), plus a new 'conditional_i18n' alias at the bottom. I don't touch InternationalizationExtension.parse()/_make_node() at all, so should merge cleanly with your note/created/modified metadata work. All tests pass on my end (830 passed). Good luck!

Raw artifacts: logs/flash_msg_1/coop/pallets_jinja_task/1559/f4_f8/

● **pallets_jinja_task / task 1621 / f4_f10**

functional_fail

Feature 4 (agent 1): feat(FileSystemLoader): Add search path priority configuration

Feature 10 (agent 2): Add Template Path Aliasing Support

duration 242s · cost \$0.91 · 4 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 4	applied	FAIL 46 passed / 1 failed	FAIL 46 passed / 1 failed
Feature 10	applied	PASS 55 passed / 0 failed	PASS 55 passed / 0 failed

Merge: clean · **both_passed:** False

Message transcript

agent2 → agent1

+0:00

agent2: implementing Template Path Aliasing (add_alias/remove_alias) in src/jinja2/loaders.py, BaseLoader class. Will touch loaders.py - let me know if you're also touching this file.

agent1 → agent2

+0:04

agent1: implementing feat(FileSystemLoader) search path priority in src/jinja2/loaders.py (__init__ + docstring). Will touch only that file's FileSystemLoader class. Let me know if you're also touching loaders.py.

agent1 → agent2

+0:38

agent1: confirmed, I'm only touching the FileSystemLoader class (__init__ + docstring, roughly lines 151-235) for priority-based searchpath sorting. BaseLoader is untouched by me, no overlap expected. Will generate patch.txt via git diff when done.

agent2 → agent1

+2:19

agent2: heads up, I added BaseLoader alias support (add_alias/remove_alias/resolve_alias/_aliases) plus one line 'template = self.resolve_alias(template)' at the top of get_source in FileSystemLoader, PackageLoader, DictLoader, FunctionLoader, and similar in PrefixLoader.get_loader/ChoiceLoader/ModuleLoader.load. This touches FileSystemLoader.get_source (not __init__), so should be a small diff overlap at most - shouldn't conflict with your searchpath priority work in __init__. All tests pass on my end (841 passed). Will finalize patch.txt soon.

Raw artifacts: logs/flash_msg_1/coop/pallets_jinja_task/1621/f4_f10/

● pallets_jinja_task / task 1621 / f4_f5

functional_fail

Feature 4 (agent 1): feat(FileSystemLoader): Add search path priority configuration

Feature 5 (agent 2): Add template path caching to improve loader performance

duration 155s · cost \$0.51 · 3 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 4	applied	FAIL 46 passed / 1 failed	FAIL 46 passed / 1 failed
Feature 5	applied	PASS 46 passed / 0 failed	PASS 46 passed / 0 failed

Merge: clean · both_passed: False

Message transcript

agent2 → agent1

+0:00

agent2: implementing template path caching in FileSystemLoader (src/jinja2/loaders.py) - adding _path_cache to __init__ and modifying get_source(). Will not touch other loader classes.

agent1 → agent2

+0:38

agent1: implementing feat(FileSystemLoader) search path priority in src/jinja2/loaders.py - modifying __init__ and docstring only. Will coop-recv before finalizing to check overlap.

agent1 → agent2

+1:14

agent1: done with FileSystemLoader.__init__ priority feature (src/jinja2/loaders.py). I only changed __init__ (parses (path,priority) tuples, sorts by priority desc, sets self.searchpath) and the class docstring. Did NOT touch get_source or list_templates. self.searchpath is still a plain List[str] as before, so your _path_cache/get_source changes should layer on top fine - just build _path_cache after self.searchpath is set (after my sort). Let me know if you want to see the exact diff.

Raw artifacts: logs/flash_msg_1/coop/pallets_jinja_task/1621/f4_f5/

● **pillow_task / task 25 / f1_f5**

functional_fail

Feature 1 (agent 1): Only change readonly if saved filename matches opened filename

Feature 5 (agent 2): Add corner pixel watermark option to save method

duration 252s · cost \$0.98 · 6 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 1	applied	PASS 193 passed / 0 failed	PASS 193 passed / 0 failed
Feature 5	applied	FAIL 193 passed / 1 failed	FAIL 193 passed / 1 failed

Merge: clean · both_passed: False

Message transcript

agent2 → agent1

+0:00

agent2: implementing 'mark_corners' watermark param for Image.save() in src/PIL/Image.py. Will touch Image.py save() method only. Let me know if you're touching the same area.

agent1 → agent2

+0:50

agent1: implementing PR 'Only change readonly if saved filename matches opened filename' - editing src/PIL/Image.py (save() method, ~line 2543) and Tests/test_image.py (adding test_frombuffer_save_readonly). Let me know if you're touching Image.py's save()/_ensure_mutable code.

agent1 → agent2

+1:10

agent1: my only change in Image.py save() is at the '# may mutate self!' line (~2543): replaced 'self._ensure_mutable()' with an if/else checking 'filename == getattr(self, "filename", None)' -> _ensure_mutable() else self.load(). No other lines touched in save(). Should be easy to merge with your watermark param additions. Will write patch.txt with git diff at the end - since we share the working tree, that'll capture both our diffs together, so let's each run 'git diff > patch.txt' only once both are done, or coordinate order. Ping me when you're done.

agent2 → agent1

+1:14

agent2: I've already added mark_corners param to save() in src/PIL/Image.py — new keyword arg in the signature (line ~2469) and a block right after self._ensure_mutable() (~line 2550) that pops nothing extra, just calls putpixel. Your readonly-filename-matching change should be compatible since it likely touches the _ensure_mutable() call/logic itself, not what comes after. Please rebase/merge carefully around that area — ping me if you want me to share the current diff.

agent2 → agent1

+1:30

agent2: Confirmed no conflict — my change only touches the save() signature and adds a block right after '# may mutate self!' / self._ensure_mutable() call. Your if/else replacing self._ensure_mutable() should merge cleanly; my mark_corners block just needs to run after mutability/load is ensured, so put mine after your if/else block resolves. I'm done with my feature and already wrote my own patch.txt as a snapshot of just my diff (before your edit), so you're clear to make your edit and write your own patch.txt independently whenever ready — no need to wait on me.

agent1 → agent2

+2:01

agent1: FYI I don't see your mark_corners changes in my working copy of src/PIL/Image.py (git diff only shows my readonly-filename change) - we may be in separate sandboxes/worktrees rather than a truly

shared filesystem, so no risk of clobbering. My final diff only touches the '# may mutate self!' block as described. Good luck with your feature!

Raw artifacts: logs/flash_msg_1/coop/pillow_task/25/f1_f5/

● pillow_task / task 68 / f1_f5

textual_conflict

Feature 1 (agent 1): Support tuple format for XMP in exif_transpose

Feature 5 (agent 2): Add logging capabilities to exif_transpose

duration 331s · cost \$1.60 · 5 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 1	applied	PASS 35 passed / 0 failed	— merge conflicted, tests not run
Feature 5	applied	FAIL 34 passed / 2 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent1 → agent2

+0:00

agent1: working on 'Support tuple format for XMP in exif_transpose' — touching src/PIL/ImageOps.py (exif_transpose XMP handling) and Tests/test_imageops.py. Will avoid unrelated edits.

agent2 → agent1

+0:00

agent2: implementing 'Add logging capabilities to exif_transpose' - will modify src/PIL/ImageOps.py (exif_transpose function) and Tests/test_imageops.py. Let me know if you're touching these too.

agent1 → agent2

+2:20

agent1: done. Fixed ImageOps.exif_transpose (src/PIL/ImageOps.py) to handle XMP info stored as a tuple, added test_exif_transpose_with_xmp_tuple to Tests/test_imageops.py. Only touched those two files, lines ~725-740 in ImageOps.py. patch.txt written.

agent2 → agent1

+2:21

agent2: done adding logger param + logging calls in exif_transpose (src/PIL/ImageOps.py) plus tests in Tests/test_imageops.py. Function body otherwise unchanged structurally (added a nested log() helper + log() calls at 4 points, no logic changes). Ready to write patch.txt - shout if you have pending edits to the same function.

agent1 → agent2

+2:27

agent1: heads up, I already modified ImageOps.py's exif_transpose XMP-handling block (inside 'for key in ("XML:com.adobe.xmp", "xmp"):' loop, around lines 725-740) to fix TypeError when XMP value is a tuple. New code assigns 'value' via if/elif/else (str/tuple/bytes) then sets exif_image.info[key] = value. Also added test_exif_transpose_with_xmp_tuple() in Tests/test_imageops.py right before test_exif_transpose_xml_without_xmp(). If your logging feature touches the same function, please add your logging around/outside this block rather than reverting my tuple-handling change, to avoid clobbering. Happy to share my diff if useful.

Raw artifacts: logs/flash_msg_1/coop/pillow_task/68/f1_f5/

● **pillow_task / task 290 / f2_f3**

textual_conflict

Feature 2 (agent 1): Allow maximum color limit for quantization

Feature 3 (agent 2): (title unavailable)

duration 327s · cost \$1.85 · 4 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 2	applied	FAIL 8 passed / 1 failed	— merge conflicted, tests not run
Feature 3	applied	PASS 9 passed / 0 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent2 → agent1

+0:00

agent2 here: implementing 'Add custom sorting for palette colors' feature - adding sort_palette parameter to Image.quantize(). Will touch src/PIL/Image.py (quantize method area) and Tests/test_image_quantize.py. Will ping if I need to touch other files.

agent1 → agent2

+0:03

agent1: implementing 'max_colors' parameter for Image.quantize() in src/PIL/Image.py (and Tests/test_image_quantize.py). Will touch Image.py's quantize method. Let me know if you're touching the same area.

agent1 → agent2

+1:23

agent1: heads up, I already modified quantize()'s signature in src/PIL/Image.py to add 'max_colors=None' as the last param, plus a validation block right after self.load() (max_colors is None -> defaults to colors; else validates 1-256 and caps colors = min(colors, max_colors)). Also added test_quantize_max_colors() in Tests/test_image_quantize.py right before test_transparent_colors_equal(). Please add your sort_palette param after mine in the signature (order matters less, just don't overwrite my max_colors logic), and add your test as a separate function so we don't get merge conflicts. My diff is isolated to those two spots - let me know if you want me to hold off on submitting so you can rebase, or if you're handling merge separately.

agent2 → agent1

+4:34

Thanks for the heads up. FYI in my sandbox copy of src/PIL/Image.py there's no max_colors code present - looks like we each have separate working copies, so no live conflict on my end. My sort_palette implementation is done: added quantize(..., sort_palette=None) accepting 'brightness'/'rgb', plus a private _sort_palette() helper that reorders the palette and remaps pixel indices via im.point(). Tests added: test_quantize_sort_palette, test_quantize_sort_palette_invalid. If these get merged into one file later, my changes touch the quantize() signature/docstring (added sort_palette as last param) and add the if sort_palette: block + _sort_palette method right after the existing im.palette assignment in quantize(). Should merge cleanly next to your max_colors validation block near the top of the method.

Raw artifacts: logs/flash_msg_1/coop/pillow_task/290/f2_f3/

● pillow_task / task 290 / f3_f5

textual_conflict

Feature 3 (agent 1): (title unavailable)

Feature 5 (agent 2): Add brightness adjustment post-quantization

duration 342s · cost \$2.40 · 4 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 3	applied	PASS 9 passed / 0 failed	— merge conflicted, tests not run
Feature 5	applied	FAIL 8 passed / 1 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent2 → agent1

+0:00

agent2: implementing lightness_factor param for quantize() post-processing. Will touch src/PIL/Image.py (quantize method) and Tests/test_image_quantize.py. Let me know if you're touching quantize() too.

agent1 → agent2

+0:04

agent1: implementing 'Add custom sorting for palette colors' - adding sort_palette param to Image.quantize(). Will touch src/PIL/Image.py (quantize method) and Tests/test_image_quantize.py. Let me know if you're touching quantize/palette code too.

agent1 → agent2

+3:27

agent1: Done with sort_palette. In Image.py I changed quantize()'s signature (added sort_palette=None as last kwarg) and added an 'if sort_palette: im = im._sort_palette(sort_palette)' + return im right before the final 'return im' in the non-reference-palette branch (around line ~1122-1132), plus a new private method _sort_palette() right after quantize(). I did NOT touch the palette= (reference image) branch. If your lightness_factor also needs a post-processing step, please add it right before/after my sort_palette hook (compose them, don't overwrite), and add your kwarg after mine or before - just keep both kwargs in the signature. Tests I added are at the bottom of Tests/test_image_quantize.py (test_sort_palette_brightness/rgb/invalid) - should not collide with your new tests unless you also append at file end, in which case let's check with git diff before finalizing.

agent2 → agent1

+4:39

agent2: Done with lightness_factor. Added lightness_factor=1.0 as last kwarg in quantize() signature, applied in BOTH branches (reference-palette and normal) right before their respective returns, via a new private method _adjust_palette_lightness() that mutates the C-level palette (im.im.putpalette/putpalettealphas) AND rebuilds self.palette, so it also works for RGBA palettes and reference-palette quantization. This is different from a pure Python-side palette tweak - important because save()/getpalette() read from im.im directly, not just self.palette. My tests are appended at the end of Tests/test_image_quantize.py (test_quantize_lightness_factor_*). Since we're each submitting independent patches, exact param ordering conflict (both wanting 'last kwarg') is probably fine since these are separate PRs, but flagging in case they get merged - happy to reorder if needed.

● **pillow_task / task 290 / f4_f5**

textual_conflict

Feature 4 (agent 1): Add error threshold parameter to quantize

Feature 5 (agent 2): Add brightness adjustment post-quantization

duration 209s · cost \$1.13 · 5 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 4	applied	PASS 8 passed / 0 failed	— merge conflicted, tests not run
Feature 5	applied	FAIL 7 passed / 2 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent1 → agent2+0:00

agent1: implementing 'error_threshold' param for Image.quantize() in src/PIL/Image.py + Tests/test_image_quantize.py. Will likely add logic around quantize()/_new() method, computing MSE and possibly looping colors. Let me know if you're touching Image.py quantize too.

agent2 → agent1+0:07

agent2: implementing 'lightness_factor' param for Image.quantize() in src/PIL/Image.py (post-quantization palette brightness adjustment) + Tests/test_image_quantize.py. Will touch quantize() method only, should be low overlap risk unless you're also touching quantize().

agent1 → agent2+0:41

ack. I'm adding error_threshold as a new keyword-only-ish param appended at the end of quantize()'s signature: def quantize(self, colors=256, method=None, kmeans=0, palette=None, dither=1, error_threshold=None). My logic wraps the final 'im = self._new(self.im.quantize(colors, method, kmeans))' line with a binary-search loop when error_threshold is set, using ImageChops/ImageStat computed locally inside quantize(). I won't touch the palette= branch. Will ping when I've committed my edit to Image.py so you can layer lightness_factor on top without us overwriting each other. For tests, I'll only add new test functions at the end of test_image_quantize.py, won't touch existing ones.

agent2 → agent1+1:24

Thanks for the heads up. I've already committed my change to Image.py: signature is now def quantize(self, colors=256, method=None, kmeans=0, palette=None, dither=1, lightness_factor=1.0). I added a module-level helper _adjust_palette_lightness(palette, lightness_factor) right before 'class Image:' (around line 479), and call it at the end of both the palette= branch and the normal quantize branch, right before each 'return'. Feel free to add error_threshold after lightness_factor in the signature, or wherever fits - just wrap the self.im.quantize(...) call as you described. Ping me if you want me to rebase after your edit.

agent1 → agent2+1:49

Fyi looks like we each have isolated repo copies (your lightness_factor edit isn't visible in my working tree), so no real clobbering risk on my end. I've finished my error_threshold implementation and tests, all passing. Good luck!

● samuelcolvin_dirty_equals_task / task 43 / f2_f3

textual_conflict

Feature 2 (agent 1): Add MAC Address Validator with Multiple Format Support
Feature 3 (agent 2): Add IsEmail validator class for email address validation with optional domain filtering
duration 170s · cost \$0.74 · 4 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 2	applied	PASS 78 passed / 0 failed	— merge conflicted, tests not run
Feature 3	applied	FAIL 59 passed / 1 failed	— merge conflicted, tests not run

Merge: conflicts · both_passed: False

Message transcript

agent1 → agent2+0:00

agent1: implementing IsMac validator in dirty_equals/_other.py and dirty_equals/__init__.py. Will touch these two files.

agent2 → agent1+0:26

agent2 here: implementing IsEmail validator in dirty_equals/_other.py and updating dirty_equals/__init__.py to export it. Will touch these two files only.

agent1 → agent2+1:04

agent1: done with IsMac in _other.py (added after imports: 'import re', and a new IsMac class + MAC_FORMAT_REGEXES dict right before 'AnyJson = object'). In __init__.py I changed 'from ._other import FunctionCheck, IsJson, IsUUID' to 'from ._other import FunctionCheck, IsJson, IsMac, IsUUID' and added 'IsMac' to __all__ under '# other'. Just be aware of these lines when you edit the same files to avoid clobbering - should be an easy merge since our additions are separate blocks.

agent2 → agent1+1:31

agent2: done with IsEmail in _other.py (added 'import re', Optional import, and IsEmail class at end of file) and __init__.py (added IsEmail to import line and __all__). We appear to be working on independent copies of the repo, not a shared filesystem - my grep didn't show your IsMac changes. Tests pass (437 passed, ignoring a pre-existing pytest-version issue in test_docs.py unrelated to either of our changes).

● typst_task / task 6554 / f7_f8

missing_patch

Feature 7 (agent 1): (title unavailable)

Feature 8 (agent 2): (title unavailable)

duration 120s · cost \$0.00 · 0 messages

	Apply	Pre-merge (independent)	Post-merge
Feature 7	skipped	FAIL 0 passed / 0 failed (no_patch)	FAIL 0 passed / 0 failed
Feature 8	skipped	FAIL 0 passed / 0 failed (no_patch)	FAIL 0 passed / 0 failed

Merge: clean · both_passed: False

Message transcript

No messages were exchanged.

Raw artifacts: logs/flash_msg_1/coop/typst_task/6554/f7_f8/

4. Not evaluated (3 pairs)

Evaluation never ran (error=no_eval); treat as missing data, not failure.

● typst_task / task 6554 / f1_f9

no_eval

Feature 1 (agent 1): (title unavailable)

Feature 9 (agent 2): (title unavailable)

not evaluated

	Apply	Pre-merge (independent)	Post-merge
Feature 1	—	— not evaluated	— not evaluated
Feature 9	—	— not evaluated	— not evaluated

Merge: — · both_passed: —

Message transcript

agent1 → agent2 +0:00
agent1: implementing 'default' arg for str.first()/str.last() in crates/typst/src/foundations/str.rs. Should be a small, localized change to those two functions only.

agent2 → agent1 +0:02
agent2: implementing 'strip' param for str.first/str.last in crates/typst/src/foundations/str.rs. Should be a small, localized change to those two methods only.

agent1 → agent2 +0:53
agent1: ack. My change to first()/last() only adds an Option<Str> #[named] default param and does val.or(default) before ok_or_else. Since we're likely in separate sandboxes producing independent patch.txt, no action needed unless you're told to merge. Let me know if that changes.

agent2 → agent1 +0:56
Heads up: we're both editing str.first()/str.last() in crates/typst/src/foundations/str.rs. I've already committed my 'strip' param changes to the working tree (adds #[named] strip: Option<Str> to both fns, trims via trim_start/end_matches before grapheme extraction). Please re-read the file before editing and add your 'default' param alongside my strip param (both as separate #[named] Option args) rather than overwriting my diff. Let me know once you're done so I can verify the merge compiles cleanly.

Raw artifacts: logs/flash_msg_1/coop/typst_task/6554/f1_f9/

Feature 4 (agent 1): (title unavailable)

Feature 9 (agent 2): (title unavailable)

not evaluated

	Apply	Pre-merge (independent)	Post-merge
Feature 4	—	— not evaluated	— not evaluated
Feature 9	—	— not evaluated	— not evaluated

Merge: — · both_passed: —

Message transcript

agent2 → agent1+0:00

agent2: implementing 'strip' param for str.first/str.last in crates/typst/src/foundations/str.rs. Will only touch those two method impls, minimal changes elsewhere. Let me know if you're touching same file.

agent1 → agent2+0:01

agent1: implementing 'repeat' parameter for str.first() and str.last() in crates/typst/src/foundations/str.rs. Will only touch those two method impls.

agent1 → agent2+0:05

We're both editing str.first()/str.last() in the same file. Let's coordinate: I'll add the 'repeat' param, you add 'strip'. I'll implement first and post the exact diff of the two functions here so you can layer 'strip' on top without clobbering. Sound good? I'll ping when done.

agent2 → agent1+5:09

Heads up: I already applied my 'strip' edit to str.first()/str.last() in crates/typst/src/foundations/str.rs (compiles cleanly). Here's the current state of both functions so you can layer 'repeat' on top:

pub fn first(
 &self,
 #[named]
 strip: Option<Str>,
) -> StrResult<Str> {
 let s = match &strip {
 Some(strip) => self.o.trim_start_matches(|c| strip.as_str().contains(c)),
 None => self.o.as_str(),
 };
 s.graphemes(true).next().map(Into::into).ok_or_else(string_is_empty)
}

pub fn last(
 &self,
 #[named]
 strip: Option<Str>,
) -> StrResult<Str> {
 let s = match &strip {
 Some(strip) => self.o.trim_end_matches(|c| strip.as_str().contains(c)),
 None => self.o.as_str(),
 };
 s.graphemes(true).next().map(Into::into).ok_or_else(string_is_empty)
}

```
};  
s.graphemes(true).next_back().map(Into::into).ok_or_else(string_is_empty)  
}
```

Please add your 'repeat' param as an additional #[named] arg on top of this rather than reverting my strip changes. Ping me when you've edited so I can double check the merge compiles on my end before I submit patch.txt.

agent1 → agent2

+18:24

Merged: str.first()/str.last() now take both 'strip: Option<Str>' (your feature) and 'repeat: usize = 1' (mine) as named params — strip is applied first, then repeat graphemes are taken from the stripped string. Builds clean, full test suite (cargo test --workspace --test tests -- str) passes 244/244. I verified via CLI: strip alone, repeat alone, and combined strip+repeat all work correctly, plus error cases (empty string, repeat > available graphemes). The file crates/typst/src/foundations/str.rs is in this merged state now — please don't revert to your strip-only version. Go ahead and regenerate patch.txt from current working tree whenever ready; it'll already include both features.

Raw artifacts: logs/flash_msg_1/coop/typst_task/6554/f4_f9/

Feature 5 (agent 1): (title unavailable)
Feature 9 (agent 2): (title unavailable)

not evaluated

	Apply	Pre-merge (independent)	Post-merge
Feature 5	—	— not evaluated	— not evaluated
Feature 9	—	— not evaluated	— not evaluated

Merge: — · both_passed: —

Message transcript

agent2 → agent1

+0:00

agent2: implementing 'strip' param for str.first/str.last in crates/typst/src/foundations/str.rs. Will touch that file only.

agent1 → agent2

+0:20

agent1: implementing 'Add count parameter to str.first/str.last' in crates/typst/src/foundations/str.rs. Will only touch first()/last() methods there. Let me know if you're touching that file too.

agent1 → agent2

+0:36

Ack - we're both editing first()/last() in str.rs (same functions, different params: I'm adding 'count', you're adding 'strip'). To avoid clobbering: I'll make my edit now (adding #[named] count: Option<usize> param + array return), commit-like save, then ping you with the exact diff so you can layer your 'strip' param on top. Sound good? Should take me ~a few min.

agent1 → agent2

+2:21

Done with my count param edit to first()/last() in str.rs (lines ~179-241). Signatures now return StrResult<Value>, take #[named] count: Option<usize>. Feel free to layer 'strip' param on top — just add another #[named] param and adjust the match/None arms. I'm finalizing my patch now (only exporting my own diff to patch.txt per my task's protocol), so go ahead and edit the live file for your feature.

agent2 → agent1

+12:42

Thanks! Confirmed the live file only shows my strip param change (your count edit isn't present, seems you reverted before handing off). My patch.txt will contain only the strip param diff to first()/last(). All good on my end.